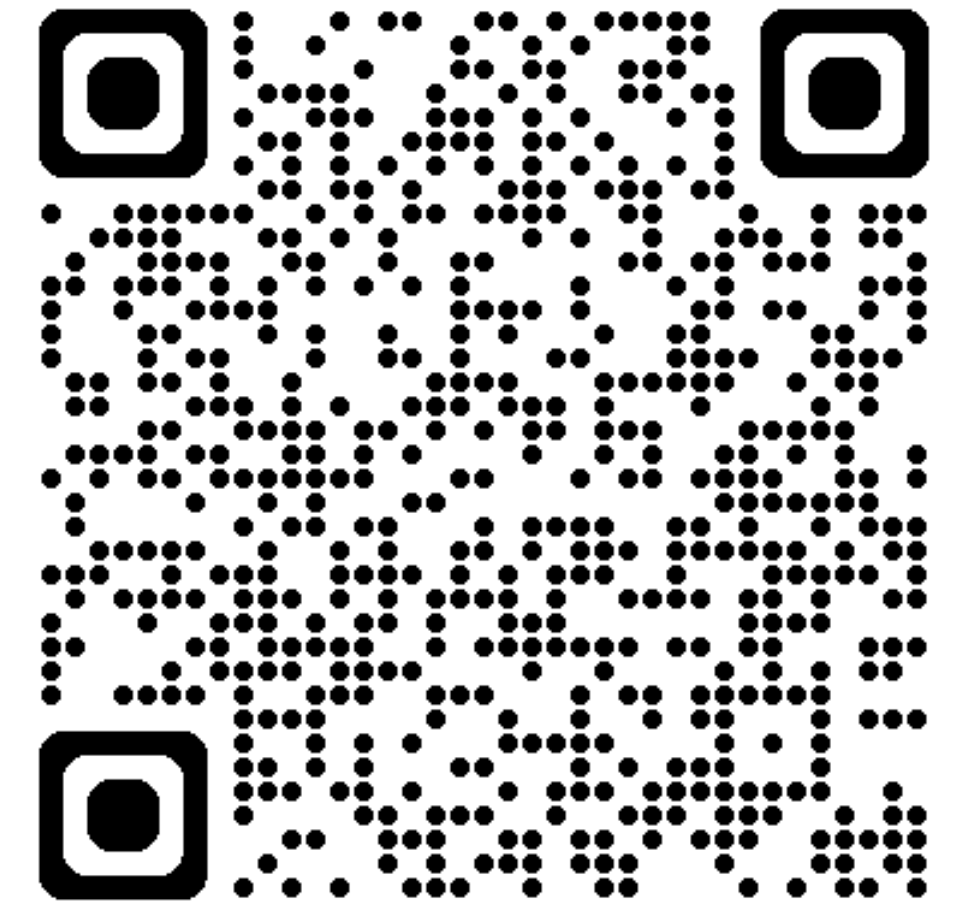


Übungsstunde 7

Heute:

- Extra: Laufzeitanalyse CodeExpert
- Repetitions Quiz
- InsertionSort verbessern
- **Rekursive Laufzeit**
- **Divide & Conquer Algorithmen**
- Ausblick CodeExpert



...Letzte Woche

Einstieg Sorting & Laufzeiten

- **Bubble Sort**
- **Insertion Sort**
- **Laufzeiten**
 - Notation
 - Schranken

danke für euer Feedback und Fragen, sowie Bemerkungen auf CodeExpert!

Gewisse Wünsche werde ich noch abdecken, ging zeitlich nicht, es auf heute vorzubereiten

Vorgehen Laufzeit bestimmen

Danke für die Anfrage danach via CodeExpert!

Ziel: anz. Aufrufe von `f()` finden.

```
def g(n):  
    for m in range(0,n):  
        for k in range(m,n):  
            f()
```

1. Wir berücksichtigen nur die innere Schleife und betrachten n und m als gegeben.
2. **TODO:** Summe Aufstellen.

Vorgehen Laufzeit bestimmen

Danke für die Anfrage danach via CodeExpert!

Ziel: anz. Aufrufe von f() finden.

```
def g(n):  
    for m in range(0,n):  
        for k in range(m,n):  
            f()
```

1. Wir berücksichtigen nur die innere Schleife und betrachten n und m als gegeben.

2. Summe Aufstellen.

$$\sum_{i=m}^n 1$$

Erklärung: Mit jeder Iteration der inneren Schleife (k) wird unsere Funktion genau einmal aufgerufen. unsere **Range** hängt von **m** und **n** ab. Entsprechend auch unsere Summe.

3. **TODO:** Summe für die Äussere scheleife finden

Vorgehen Laufzeit bestimmen

Danke für die Anfrage danach via CodeExpert!

Ziel: anz. Aufrufe von `f()` finden

3. Summe für die äußere Schleife finden

```
def g(n):  
    for m in range(0,n):  
        for k in range(m,n):  
            f()
```

Wir interessieren uns nur dafür, wie oft wir die äußere Schleife iterieren. Dementsprechend wie oft der **gesamte grüne Block** ausgeführt wird.

eine Schleife mit `range(0,n)` wird genau **n Mal** ausgeführt.

Vorgehen Laufzeit bestimmen

Danke für die Anfrage danach via CodeExpert!

```
def g(n):  
    for m in range(0,n):  
        for k in range(m,n):  
            f()
```

$$\sum_{m=1}^n \left[\sum_{i=m}^n 1 \right]$$

1, einen Funktionsaufruf pro Iteration von k

Den blauen Block (1) rufen wir innerhalb des grünen Blocks n-m Mal auf!

den grünen Block rufen wir dank der **äussersten Schleife** n Mal auf!

Vorgehen Laufzeit bestimmen

Danke für die Anfrage danach via CodeExpert!

```
def g(n):  
    for m in range(0, n):  
        for k in range(m, n):  
            f()
```

$$\sum_{m=1}^n \sum_{i=m}^n 1$$

wir können die Summe umschreiben zu:

$$\sum_{m=1}^n (n - m)$$

da wir das können wir entweder direkt oder mit der ZF ablesen dass es n^2 wird.

Analyse:

- `f()` wird so lange aufgerufen, bis das Quadrat von `m` nicht mehr kleiner ist als `n`.
- wir erhöhen `m` nach jedem aufruf von `f` um `+1`

```
def g(n):  
    m = 0;  
    while m*m < n:  
        f()  
        m = m + 1
```

Weg zur Lösung: Wie könnte eine Frage lauten, die mich auf die richtige Spur bringt?

Weg zur Lösung: Wie könnte eine Frage lauten, die mich auf die richtige Spur bringt?

```
def g(n):  
    m = 0;  
    while m*m < n:  
        f()  
        m = m + 1
```

Vorschläge:

Mathematisch: Wie gross muss m werden, damit die Bedingung nicht mehr erfüllt ist? => **sqrt(n)**

Verhalten: Wenn ich n Verdopple, wie wirkt sich dies auf die Aufrufe aus?

=> Ich quadriere jeweils m , daher erreiche ich ein doppelt so grosses n mit weniger als doppeltem M .

Weitere Fragen/ konkrete Aufgaben?

Gerne können wir weitere Beispiele zusammen besprechen, es gibt weitaus weniger relevante Themen!

Warm-up:

OOP

```
class Vehicle:
    def __init__(self, initial_position=0):
        """Initializes the vehicle at a given position (default is 0)."""
        self.position = initial_position

    def drive(self, duration):
        """Moves the vehicle based on a multiplier of 5."""
        self.position += 5 * duration

class RaceCar(Vehicle):
    def drive(self, duration):
        """Overrides the drive method to move at a multiplier of 10."""
        self.position += 10 * duration

car = Vehicle()
fast_car = RaceCar()
headstart_car = Vehicle(100)

car.drive(20)
fast_car.drive(12)
headstart_car.drive(-20)

print(car.position, fast_car.position, headstart_car.position)
print(headstart_car.position >= (car.position + fast_car.position) )
```


Warm-up

Sorting review

```
ls= [9,5,1,6,8]
```

nach der **ersten Iteration** (1-indexiert) BubbleSort:

Bestimme $ls[-1]$ und $ls[0]$

Insertion Sort

Inplace anstatt mit zweiter Liste

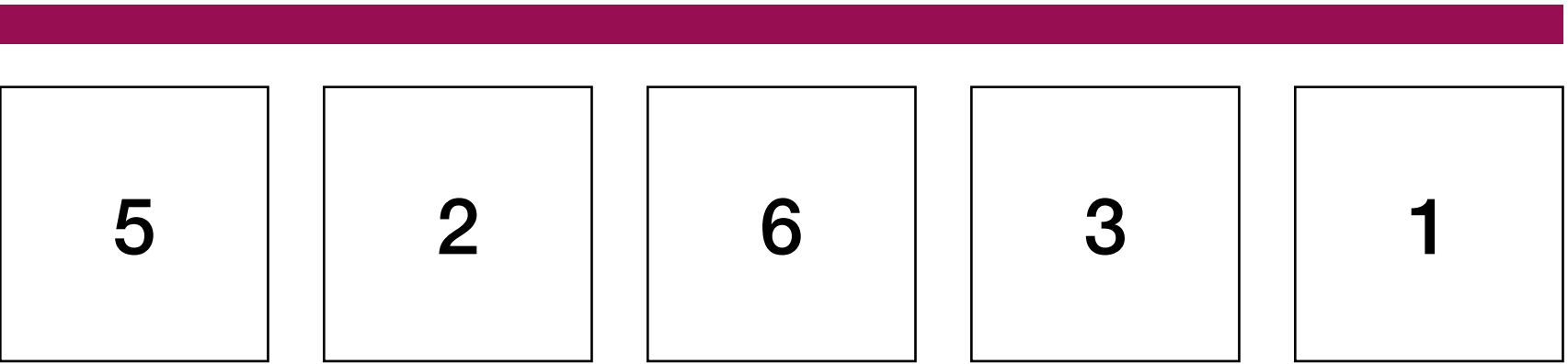
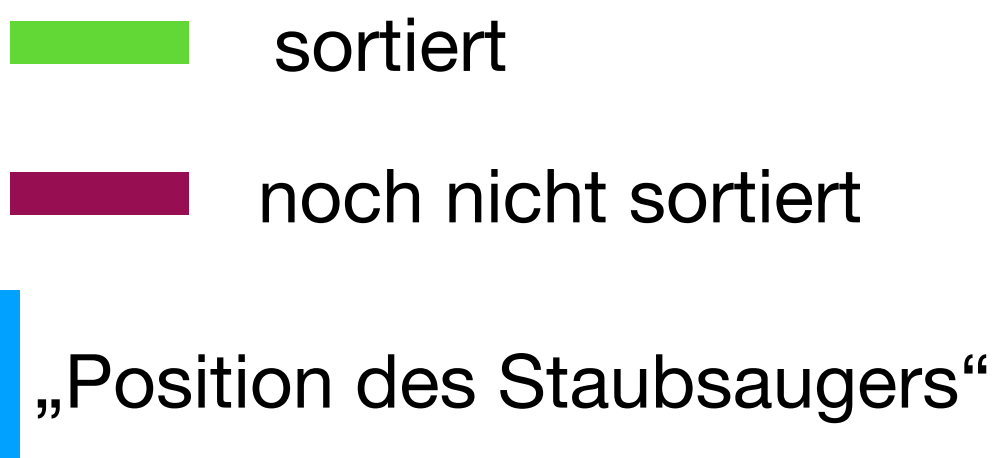


Sortiert

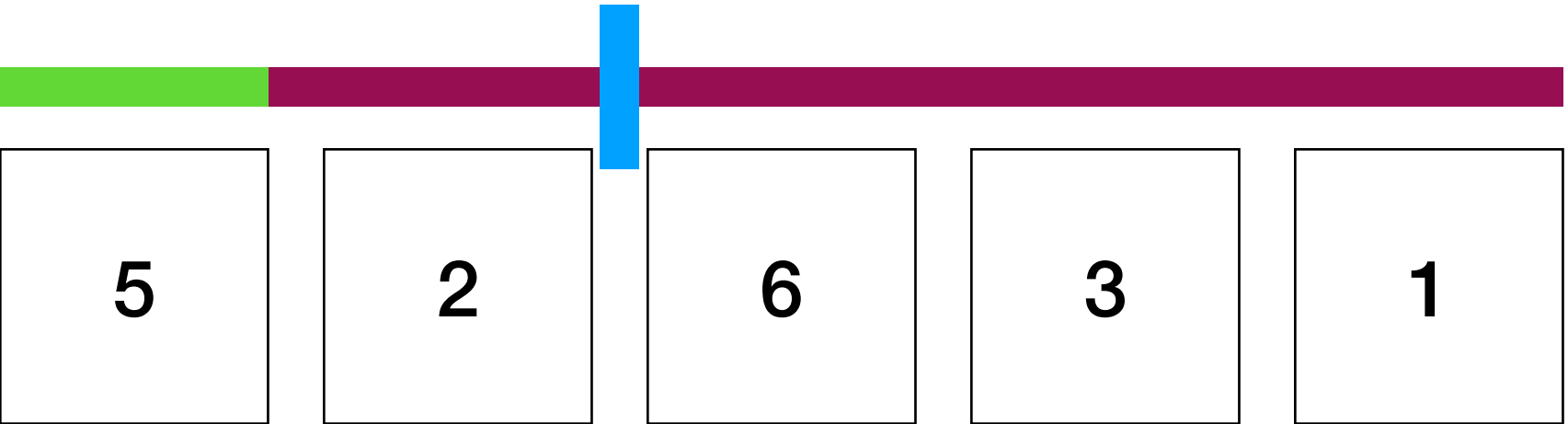
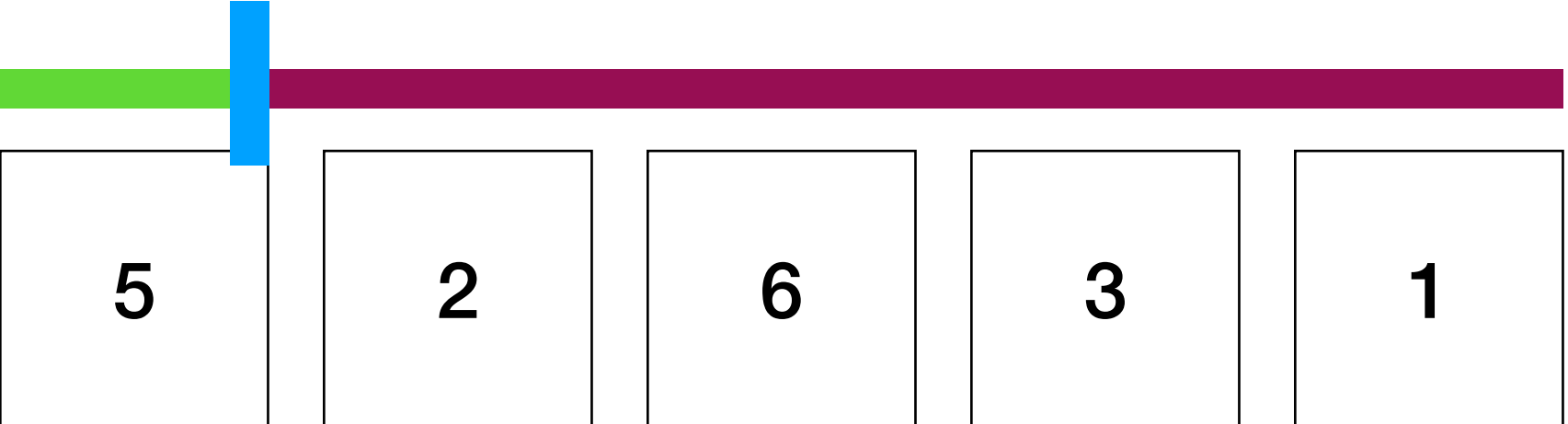
noch nicht sortiert



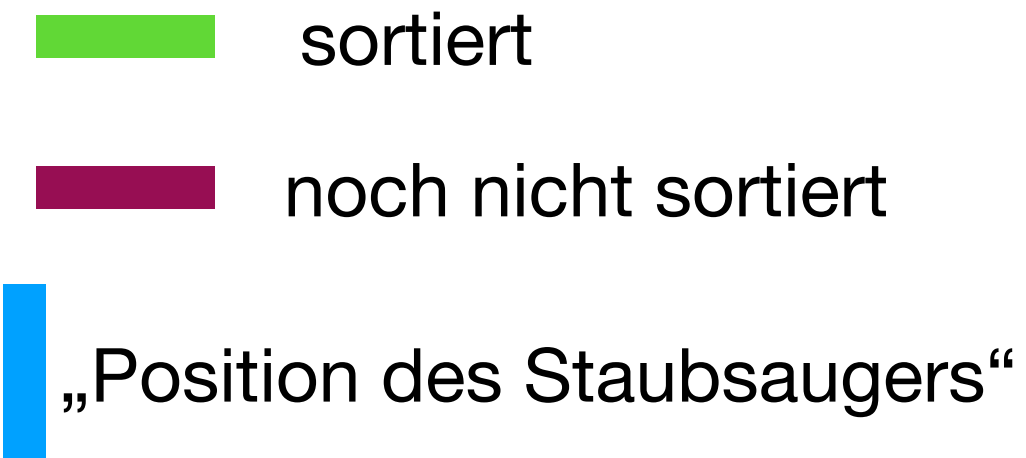
Das Vorgehen



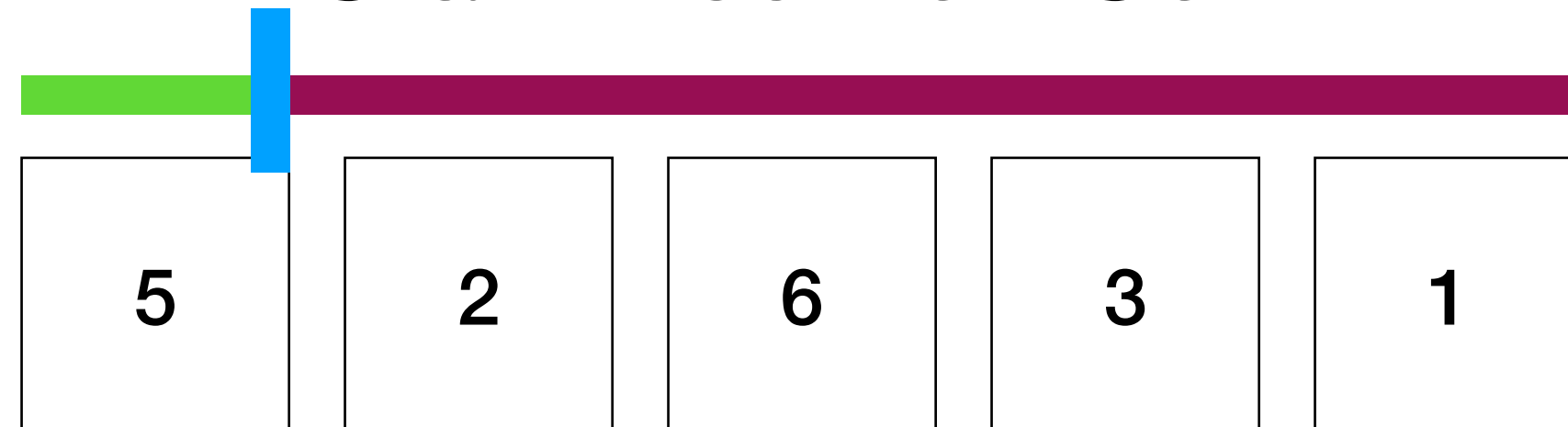
Start Insertion Sort



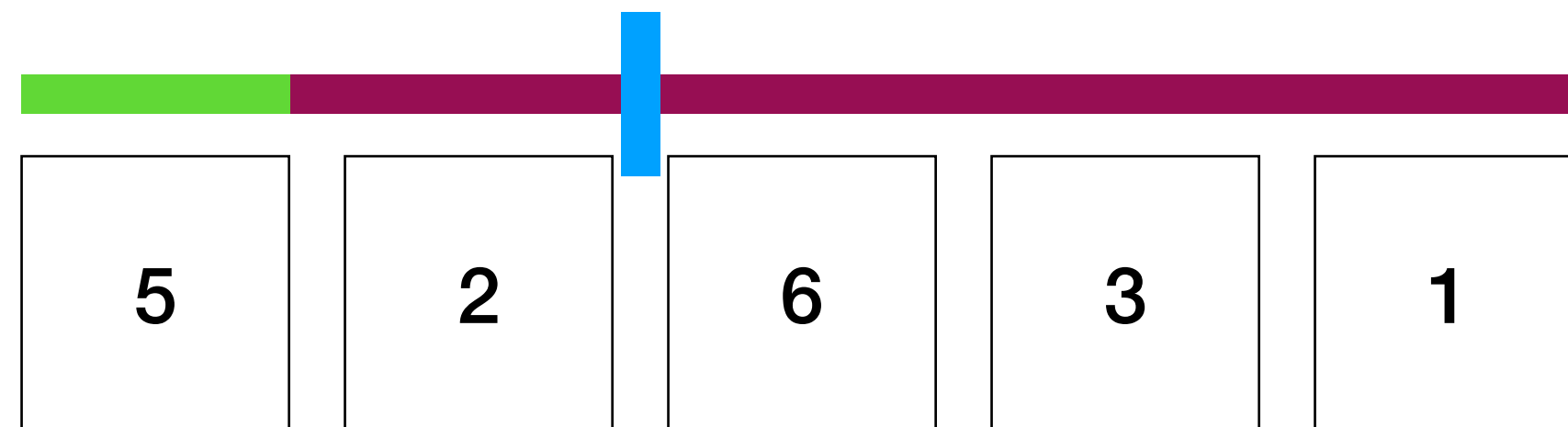
Das Vorgehen



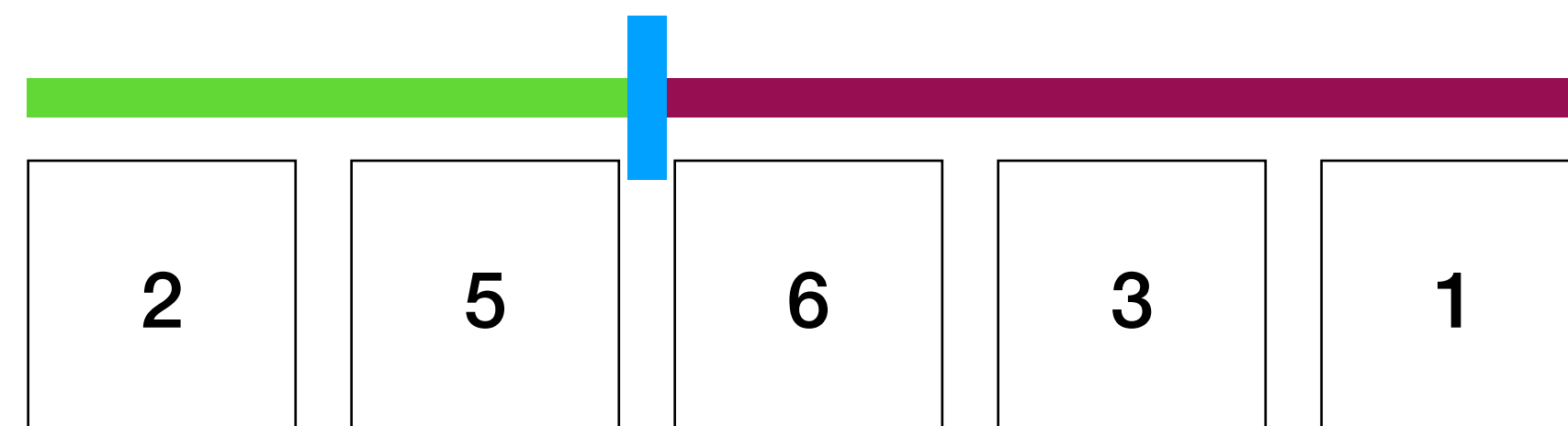
Start Insertion Sort



das erste Element ist Automatisch „sortiert“ (es steht innerhalb des grünen Bereichs korrekt)



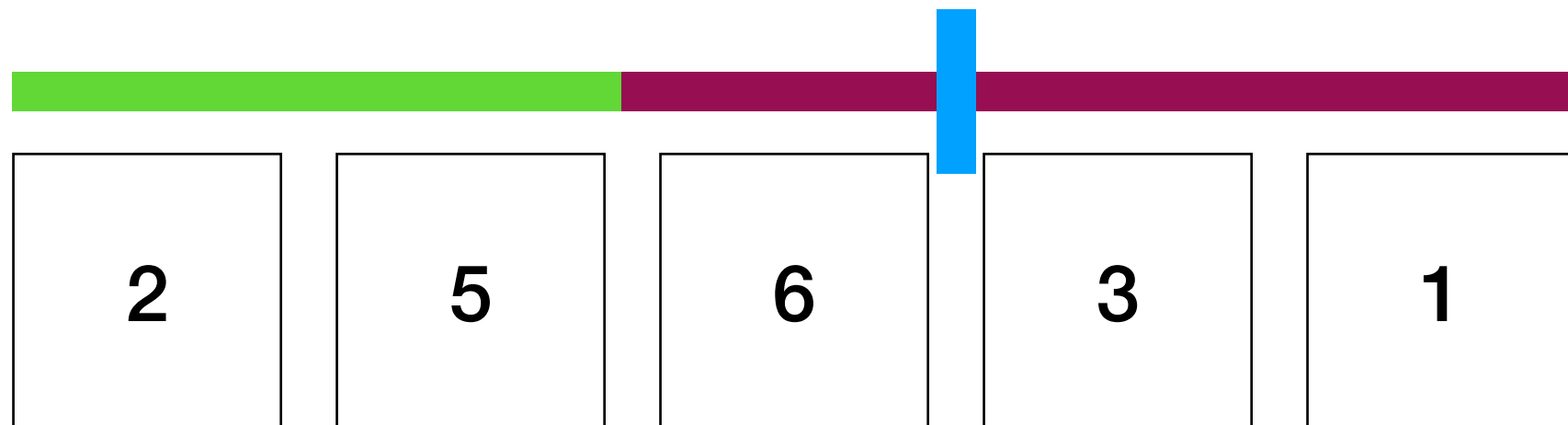
Wir platzieren nun die zwei an der passenden Stelle innerhalb des grünen Bereichs.



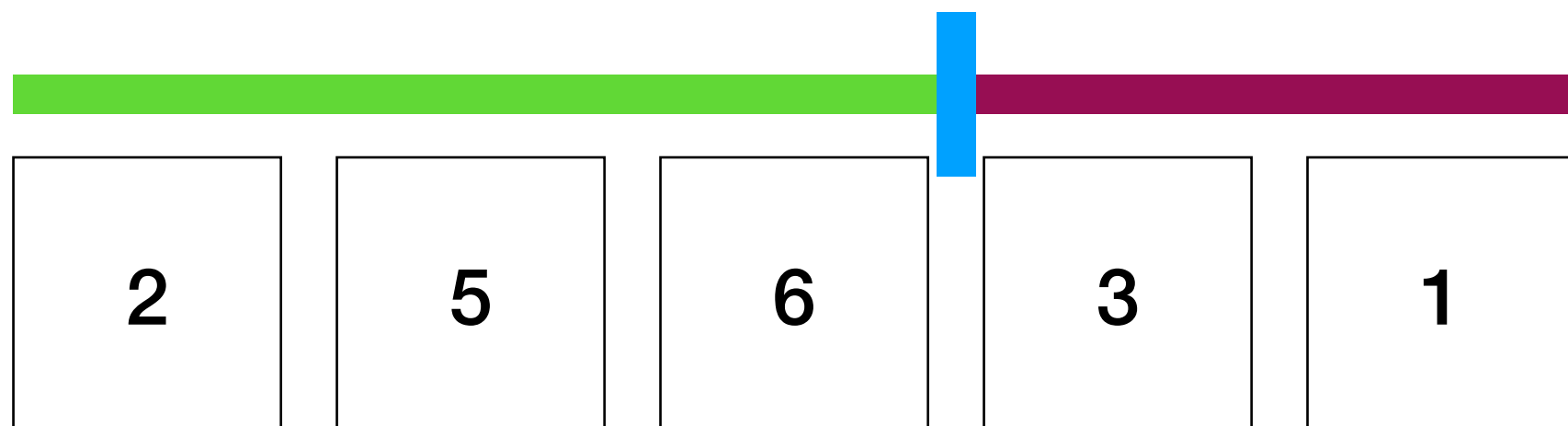
In diesem Fall genügt ein einfaches Tauschen

Das Vorgehen

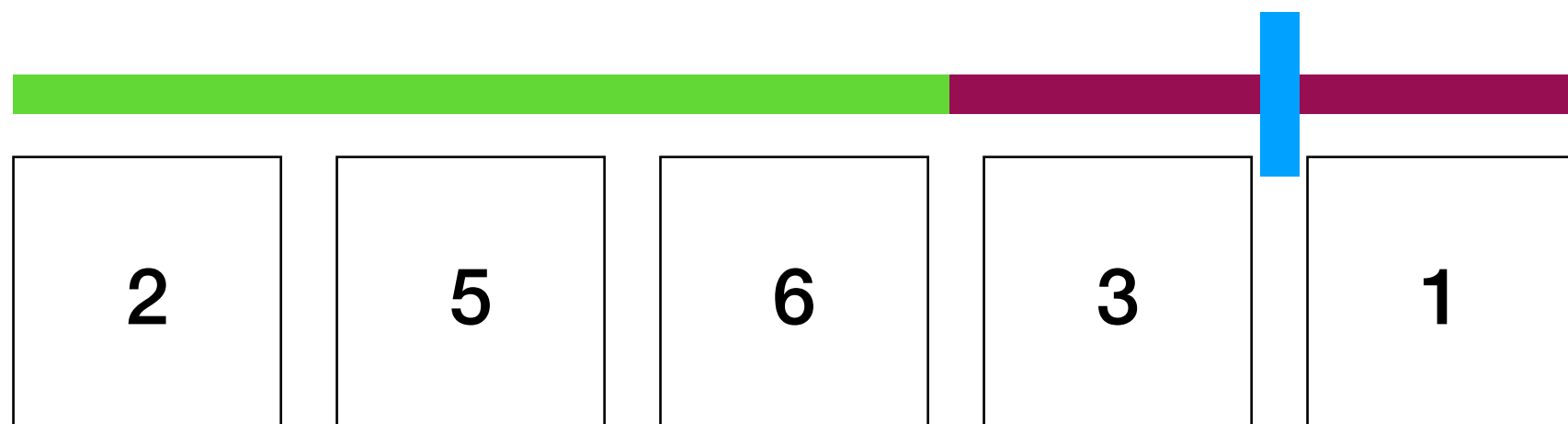
■ sortiert
■ noch nicht sortiert
■ „Position des Staubsaugers“



nun kümmern wir uns um die 6

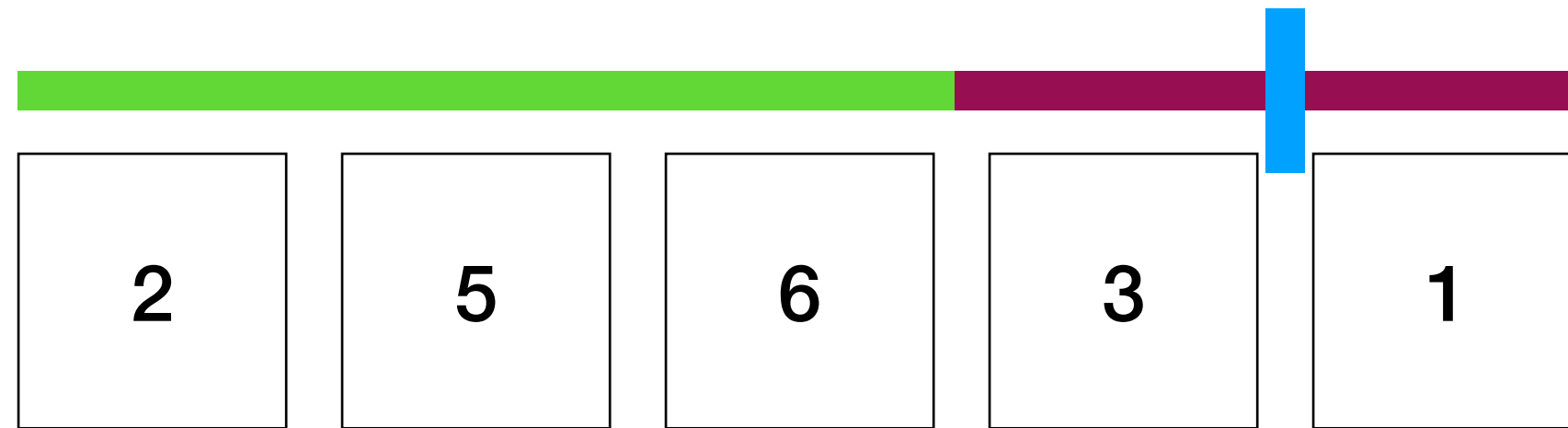
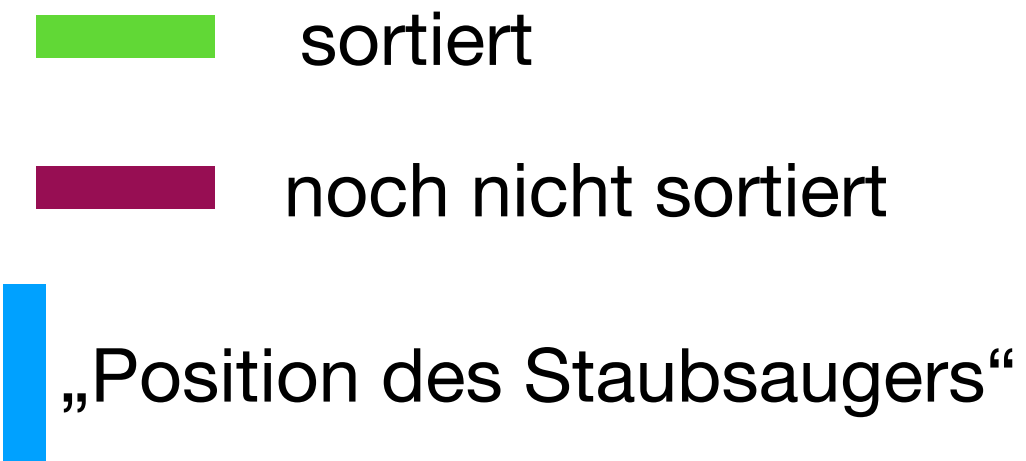


wir merken, dass Sie bereits korrekt steht, also verlängern wir das Grüne einfach.



weiter mit der Drei, nun wird es spannend.

Das Vorgehen



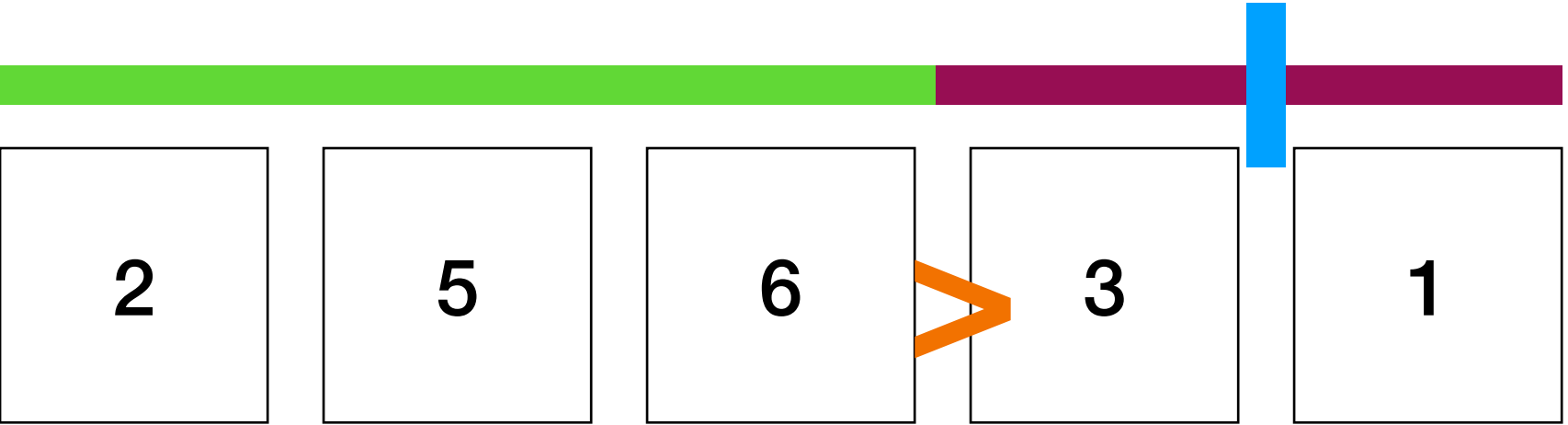
weiter mit der drei, nun wird es spannend.

um die Drei an die korrekte Stelle im Grünenbereich zu bringen, gibt es verschiedene Wege.

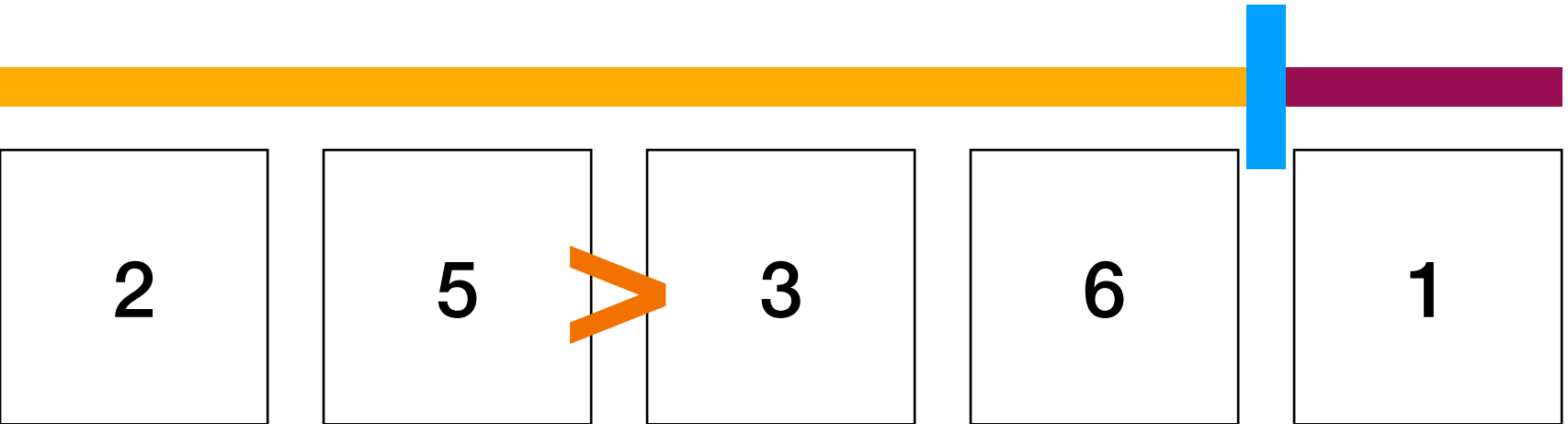
Wir wenden BubbleSort-„Wandern“ an und swappen die drei bis zur richtigen Position durch. (Standard)

BubbleSort-Swappen

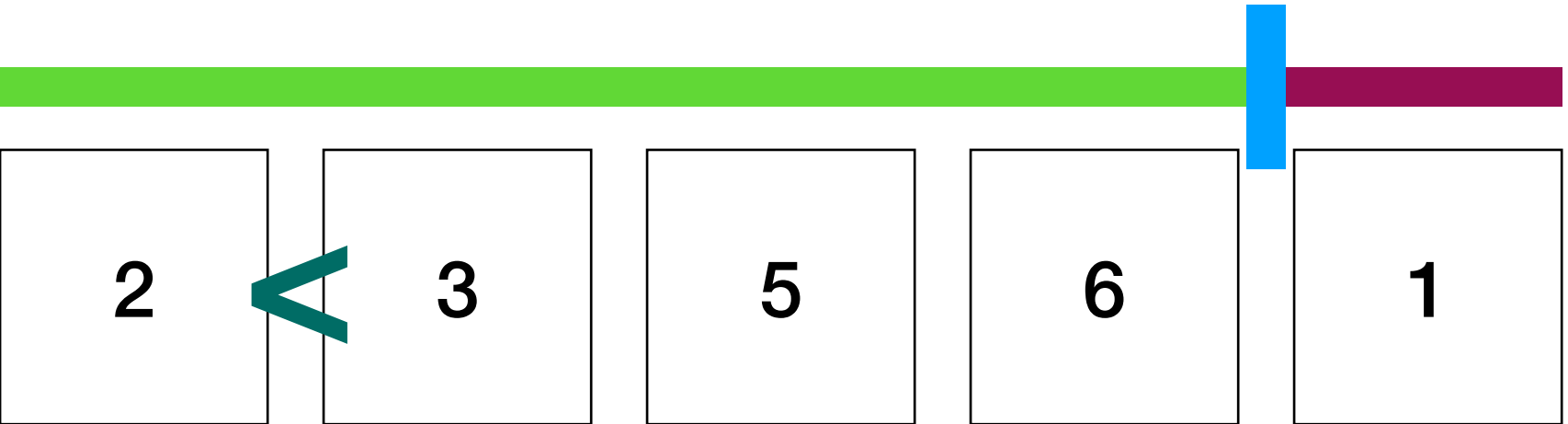
 sortiert
 noch nicht sortiert
 „Position des Staubsaugers“



swappen
mit der 6*

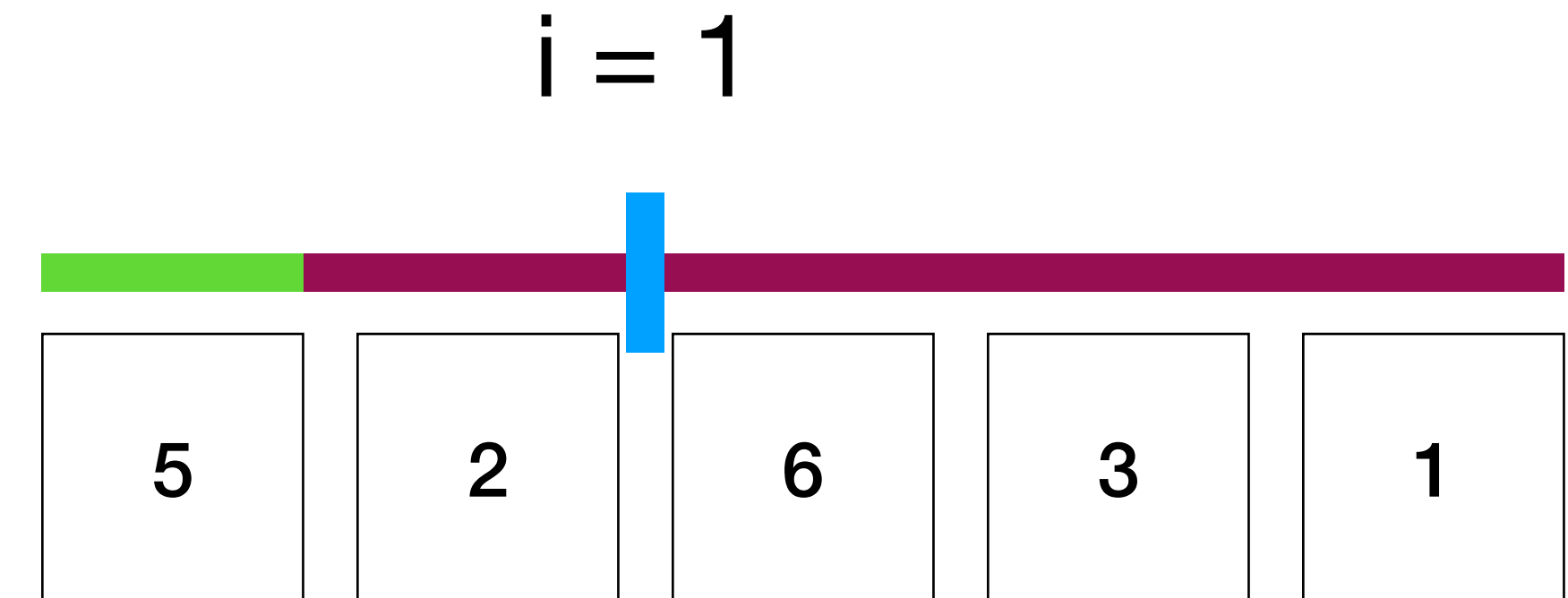


swappen



mit der 1 würde es von hier gleicher massen gemacht werden...

InsertionSort im Code



```
def insertion_sort(items):  
    n = len(items)  
    for i in range(1, n):  
        j = i  
  
        while j > 0 and items[j-1] > items[j]:  
            items[j], items[j-1] = items[j-1], items[j]  
            j -= 1
```

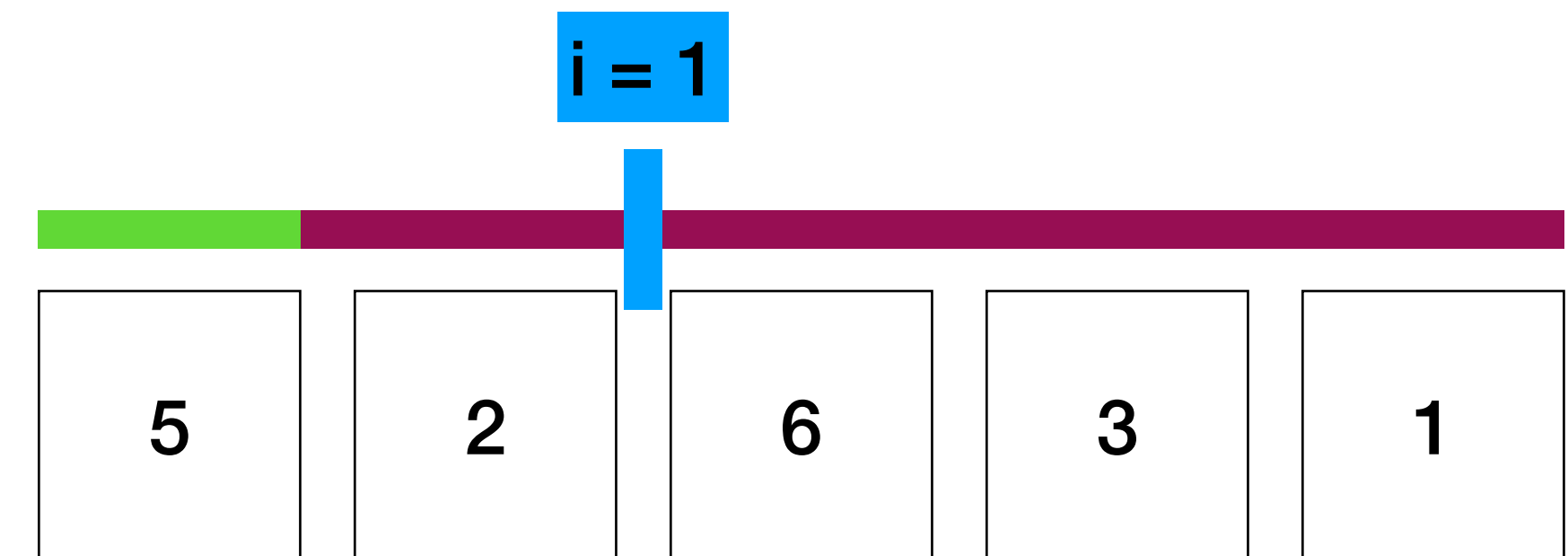
Über die Liste iterieren,
beginnen bei 1 anstatt 0 denn
das erste Element ist ja bereits
sortiert 💡

i ist der „Staubsauger“

InsertionSort im Code

```
def insertion_sort(items):  
    n = len(items)  
    for i in range(1, n):  
        j = i  
        while j > 0 and items[j-1] > items[j]:  
            items[j], items[j-1] = items[j-1], items[j]  
            j -= 1
```

(angenommen i=1)



Wie bei Bubble!

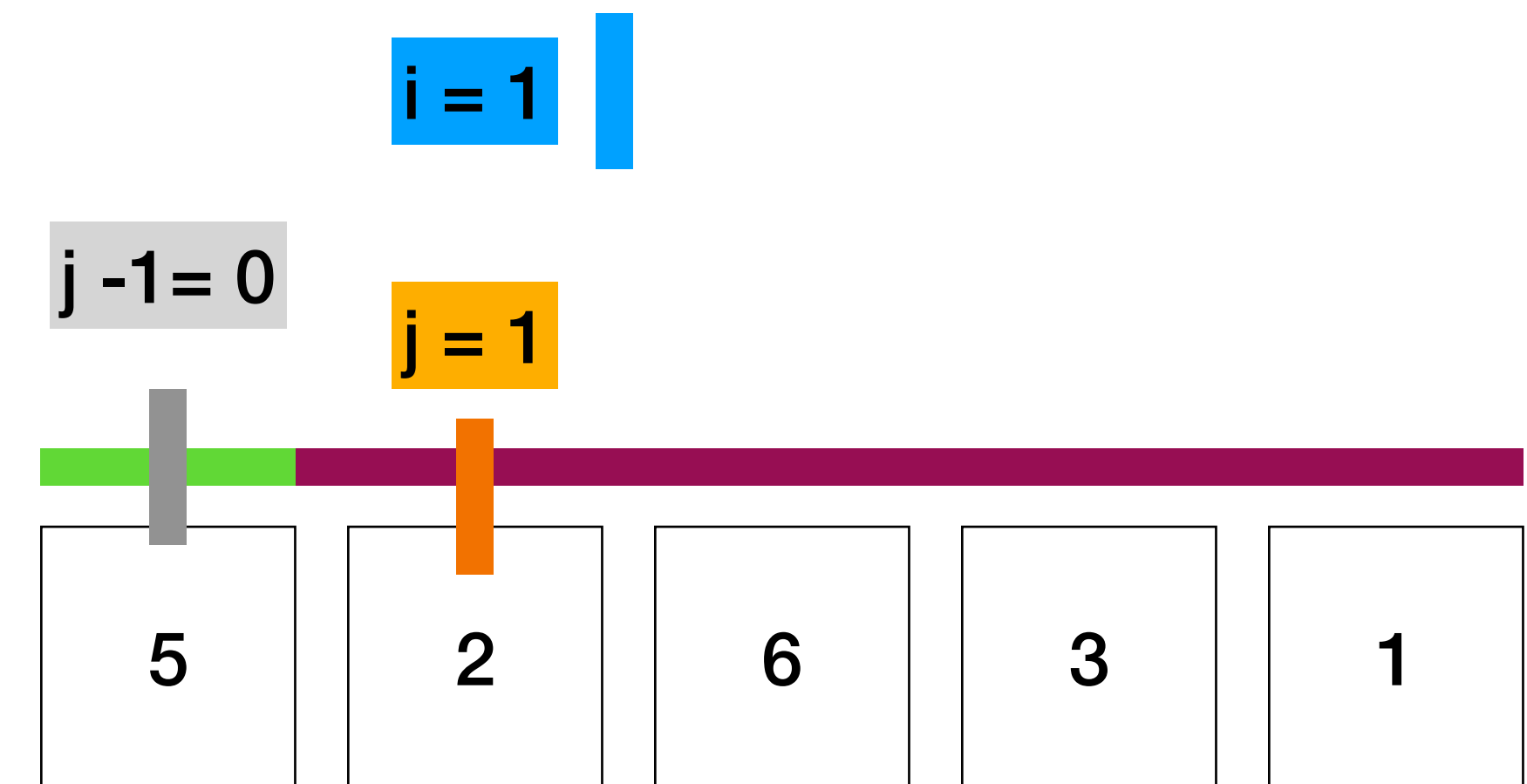
wir tauschen, solange die Bedingungen in while nicht mehr erfüllt sind!

Die Bedingungen sind:

$j > 0$ Index-Sicherheit (trifft zu für ein kleineres Element)

„Das Element links grösser ist, als das einzuordnende Element (2).“

InsertionSort im Code



die **while** schleife:

j=1

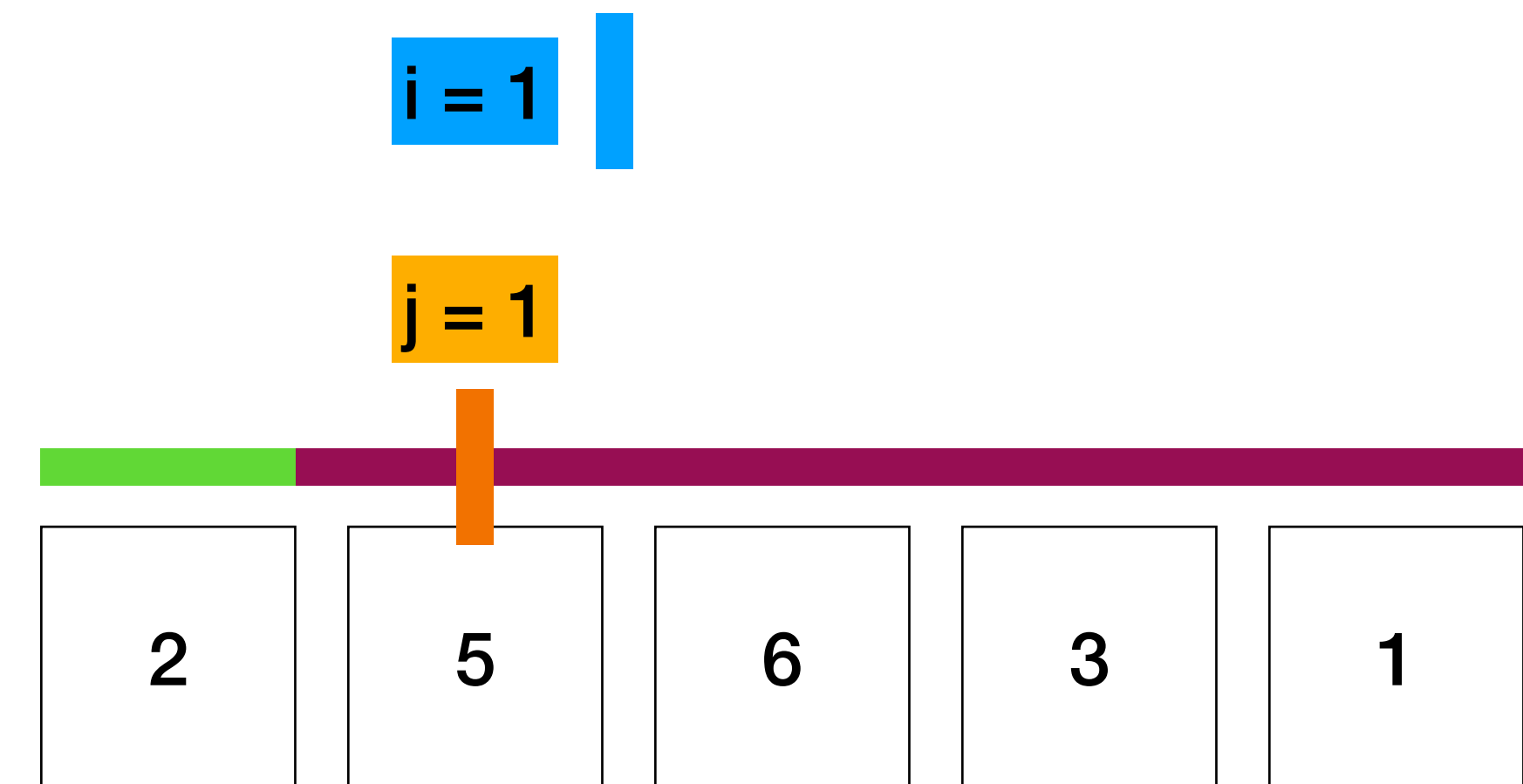
items[1]= **einzuordnendes Element** (2)

items[j-1] = 5

SWAP

```
def insertion_sort(items):  
    n = len(items)  
    for i in range(1, n):  
        j = i                                (angenommen i=1)  
  
        while j > 0 and items[j-1] > items[j]:  
            items[j], items[j-1] = items[j-1], items[j]  
            j -= 1
```

InsertionSort im Code



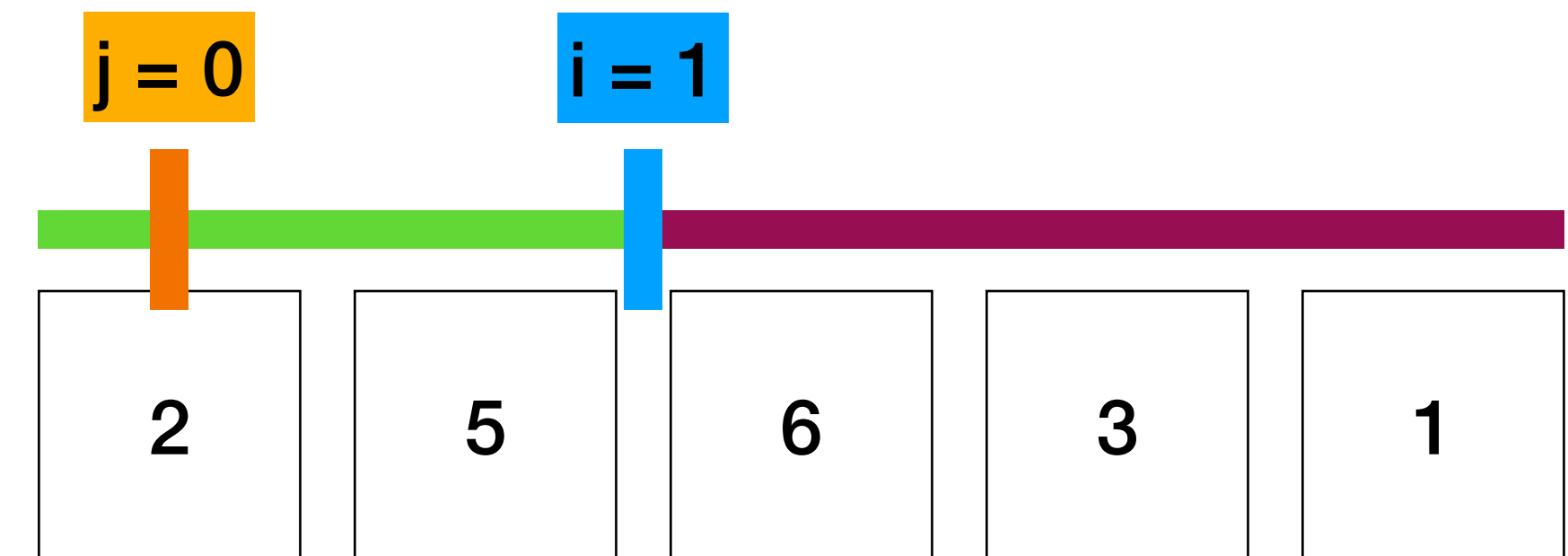
```
def insertion_sort(items):  
    n = len(items)  
    for i in range(1, n):  
        j = i                                (angenommen i=1)  
        while j > 0 and items[j-1] > items[j]:  
            items[j], items[j-1] = items[j-1], items[j]  
            j -= 1
```

Swap durchgeführt.

$j -= 1 \Rightarrow j = 0$

... while schleife $\Rightarrow$ wieder nach oben...

InsertionSort im Code



```
def insertion_sort(items):  
    n = len(items)  
    for i in range(1, n):  
        j = i                                (angenommen i=1)  
        while j > 0 and items[j-1] > items[j]:  
            items[j], items[j-1] = items[j-1], items[j]  
            j -= 1
```

... while schleife=> wieder nach oben...

j ist nicht grösser als 0.

wir sind fertig mit dem Bubble-Wandern.

Warm-up

Sorting review

```
ls= [9,5,1,6,8]
```

```
def insertion_sort(items):  
    n = len(items)  
    for i in range(1, n):  
        j = i  
        # Wir tauschen das Element so lange nach links,  
        # wie das linke Element größer ist  
        while j > 0 and items[j-1] > items[j]:  
            # Das ist der "Bubble Swap"  
            items[j], items[j-1] = items[j-1], items[j]  
            j -= 1  
        print(i, items)
```

wenn i=2 ist, was wird über print() ausgegeben?

In welchem Bereich gilt die Liste als Sortiert?


```
def insertion_sort(items):  
    n = len(items)  
    for i in range(1, n):  
        j = i  
        # Wir tauschen das Element so lange nach links,  
        # wie das linke Element größer ist  
        while j > 0 and items[j-1] > items[j]:  
            # Das ist der "Bubble Swap"  
            items[j], items[j-1] = items[j-1], items[j]  
            j -= 1  
        print(i, items)
```

Wie viele Vergleiche machen wir im schlechtesten Fall?

Wie viele Swaps machen wir im schlechtesten Fall?

```
def insertion_sort(items):  
    n = len(items)  
    for i in range(1, n):  
        j = i  
        # Wir tauschen das Element so lange nach links,  
        # wie das linke Element größer ist  
        while j > 0 and items[j-1] > items[j]:  
            # Das ist der "Bubble Swap"  
            items[j], items[j-1] = items[j-1], items[j]  
            j -= 1  
        print(i, items)
```

Wie viele Vergleiche machen wir im schlechtesten Fall? **n^2**

Wie viele Swaps machen wir im schlechtesten Fall? **n^2**

Wie könnten wir Vergleiche einsparen?

Vergleiche Sparen durch Binary Search

Bisher sind wir von rechts nach links durch den **grünen Teil** gegangen, um das nächste Element einzusetzen.

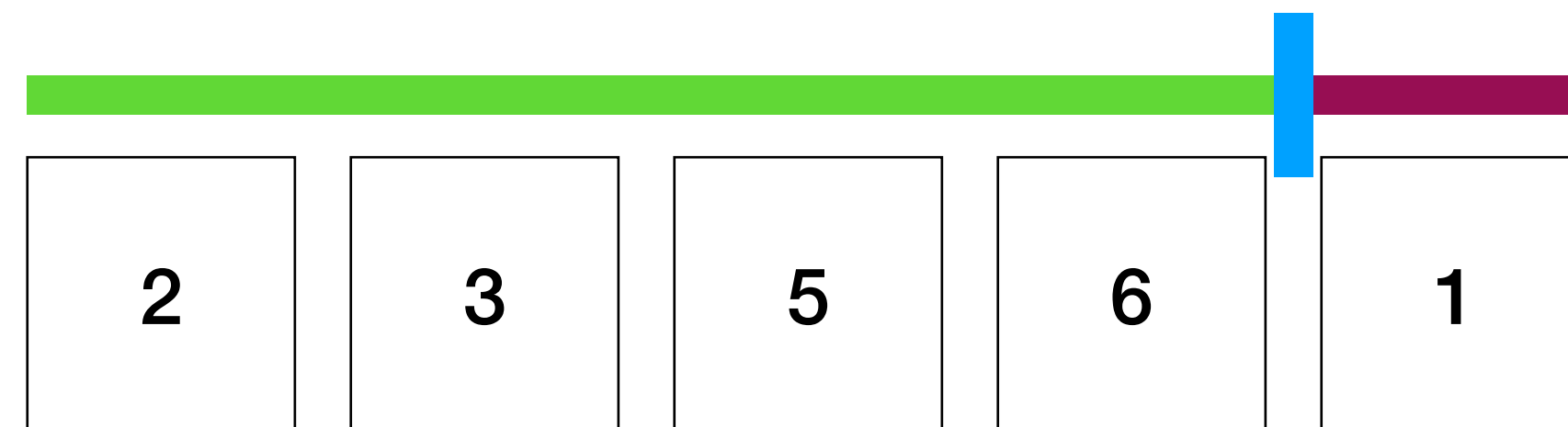
Wir haben aber noch ein **Ass im Ärmel**: Wir wissen ja, dass der grüne Teil sortiert sein muss. Mit **Binary Search** können wir schneller darin navigieren!



genau das werdet ihr in der nächsten CX-Aufgabe implementieren müssen.

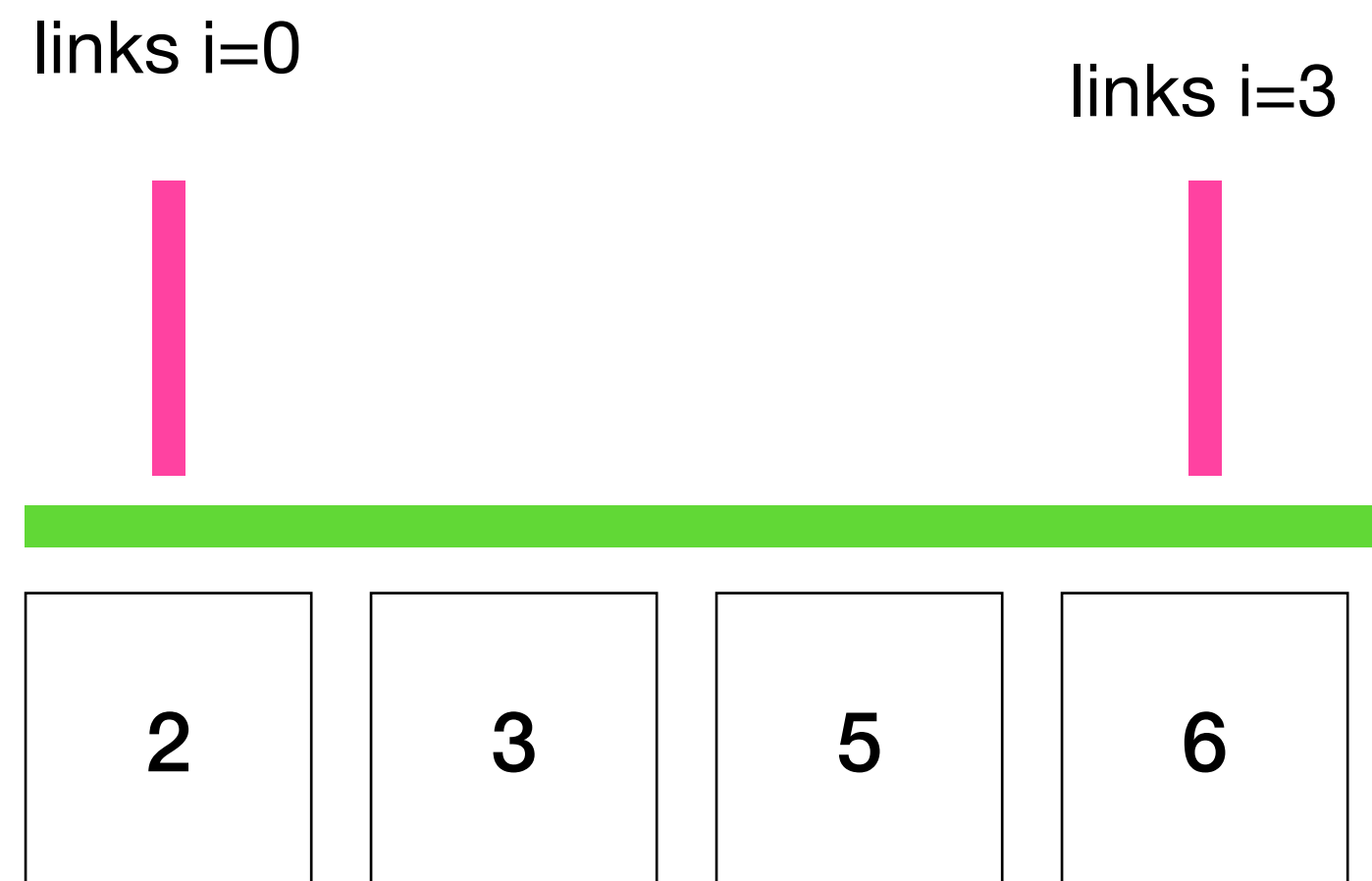
Vergleiche Sparen durch Binary Search

Einsortieren der Eins

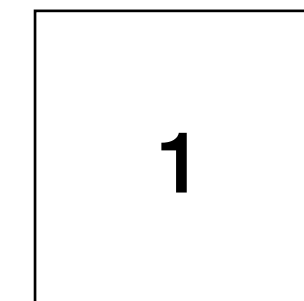


Vergleiche Sparen durch Binary Search

Einsortieren der Eins



der sortierte Teil



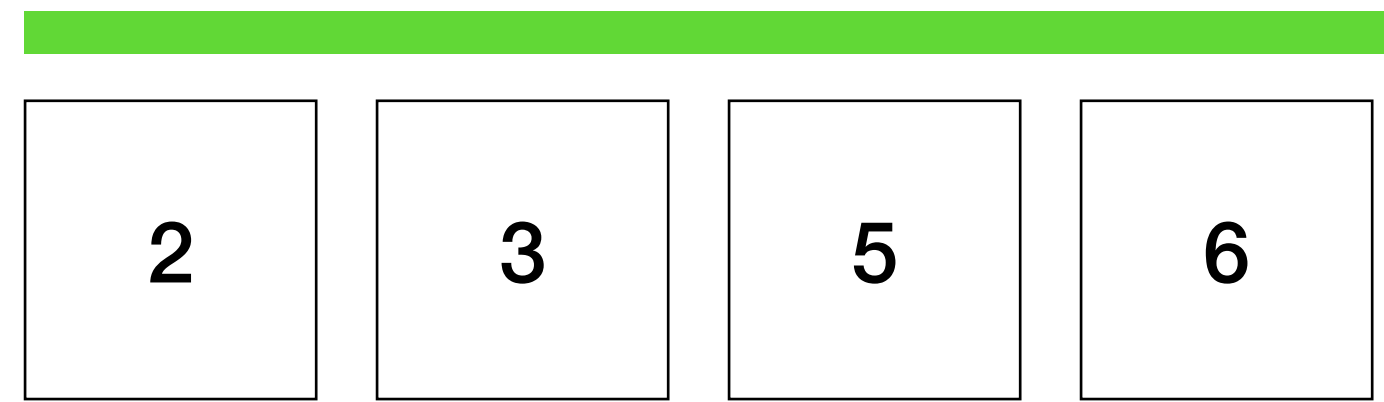
vorläufig parkiert

die linke und rechte begrenzung wir suchen nur in diesem Bereich (sortiert)

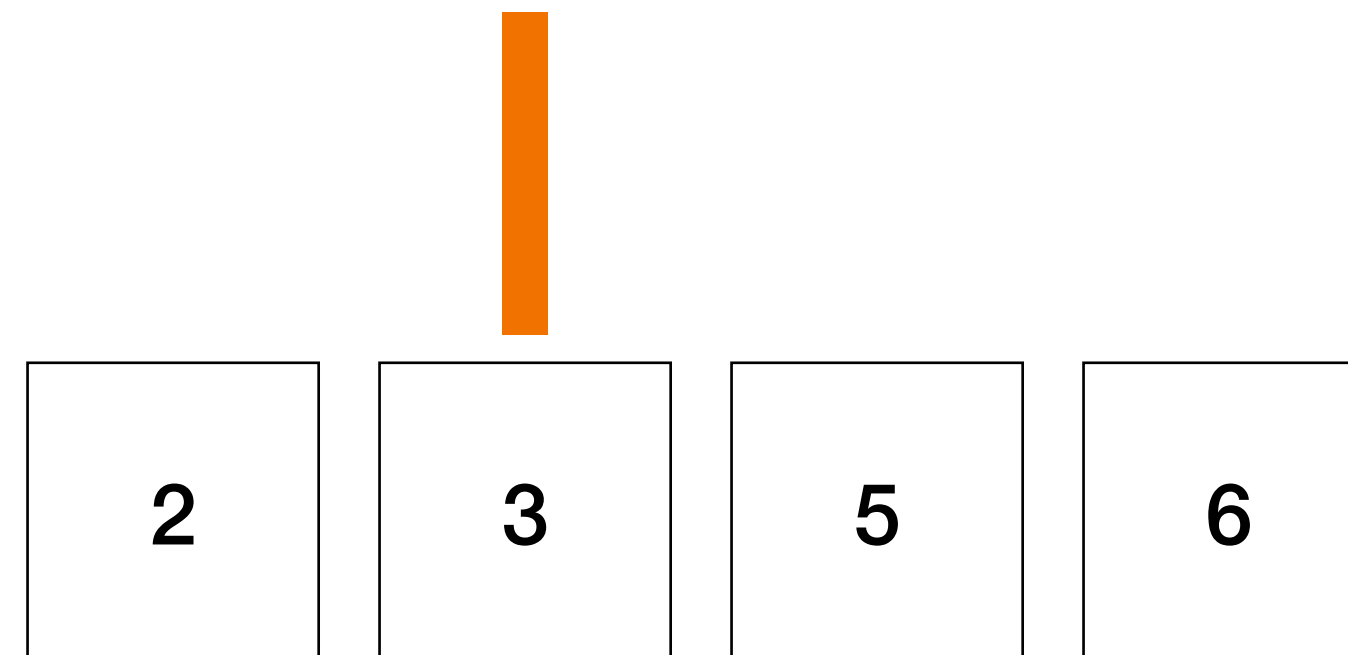
Vergleiche Sparen durch Binary Search

Finden des Indexes zum Einsetzen

der sortierte Teil



Mitte nehmen



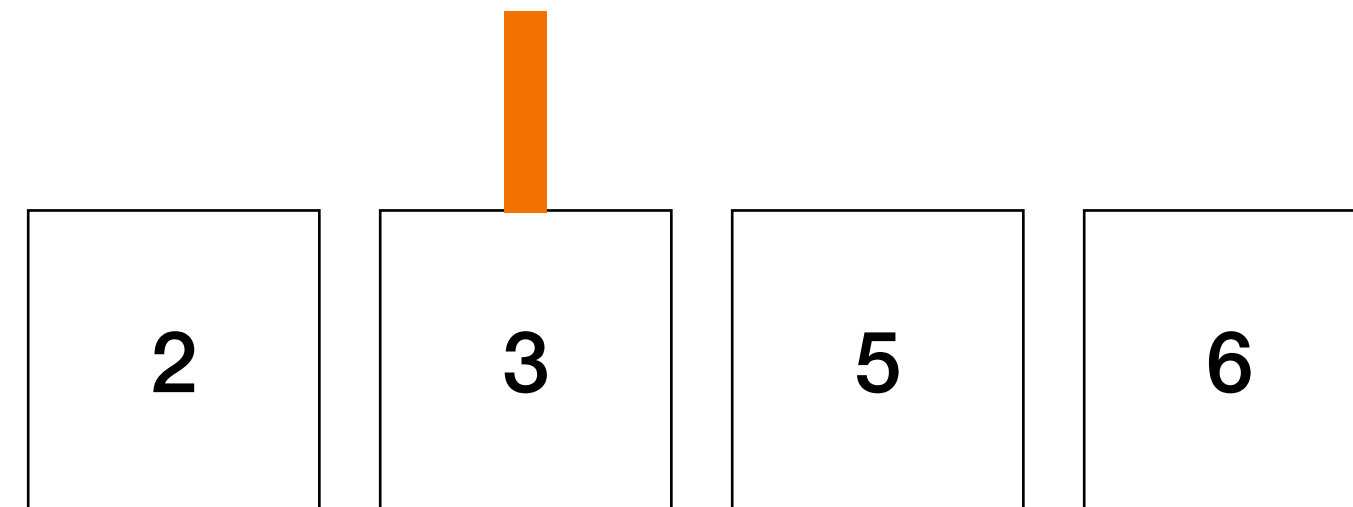
tipp: $(\text{links} + \text{rechts}) // 2$

hier: $(0 + 3) // 2 =$ **1**

Vergleiche Sparen durch Binary Search

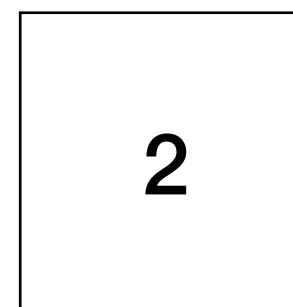
Finden des Indexes zum Einsetzen

Mitte nehmen

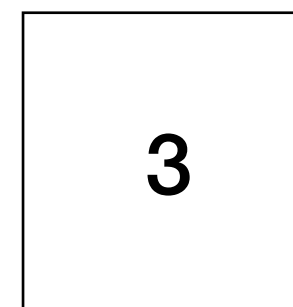


Gedanklich in 3 Segmente zerlegen: Links, Mitte und Rechts

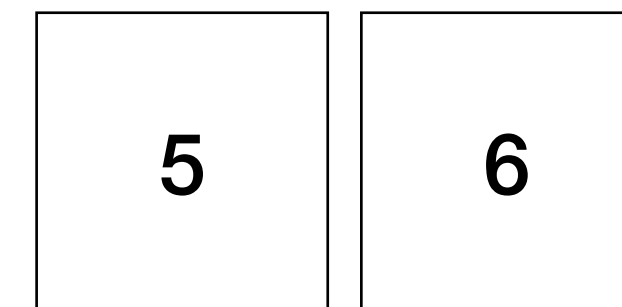
Links



Mitte



Rechts

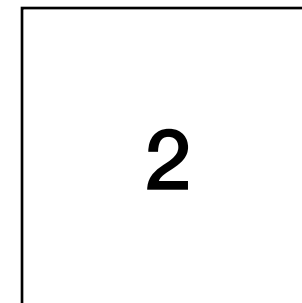


Vergleiche Sparen durch Binary Search

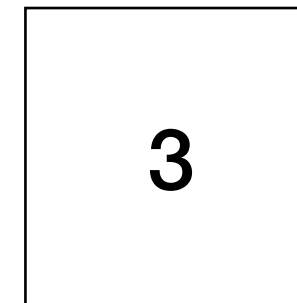
Finden des Indexes zum Einsetzen

Gedanklich in 3 Segmente zerlegen: Links, Mitte und Rechts

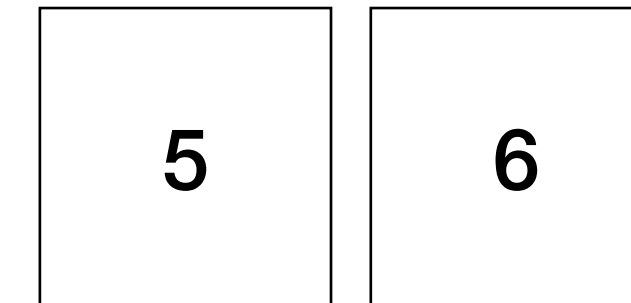
Links



Mitte



Rechts



herausfinden, in **welches dieser 3** Segmente die 1 gehört

wir **prüfen** standardmässig **die Mitte**, eventuell haben wir Glück und die Mitte hat denselben Wert.

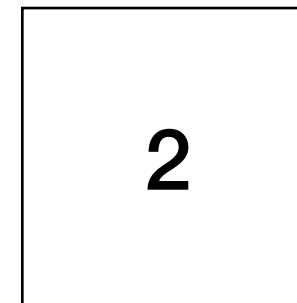
3!= 1 kein glück Gehabt diesmal :(

Vergleiche Sparen durch Binary Search

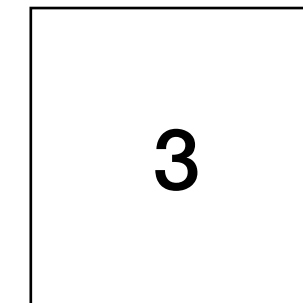
Finden des Indexes zum Einsetzen

Gedanklich in 3 Segmente zerlegen: Links, Mitte und Rechts

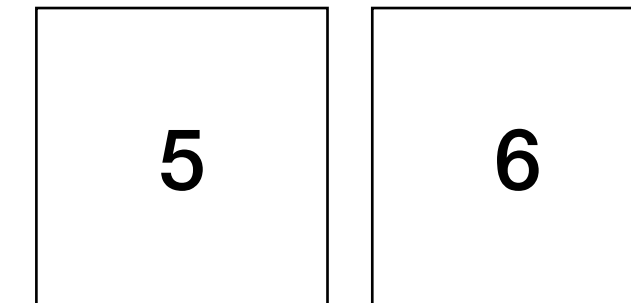
Links



Mitte



Rechts



herausfinden, in **welches dieser 3** Segmente die 1 gehört

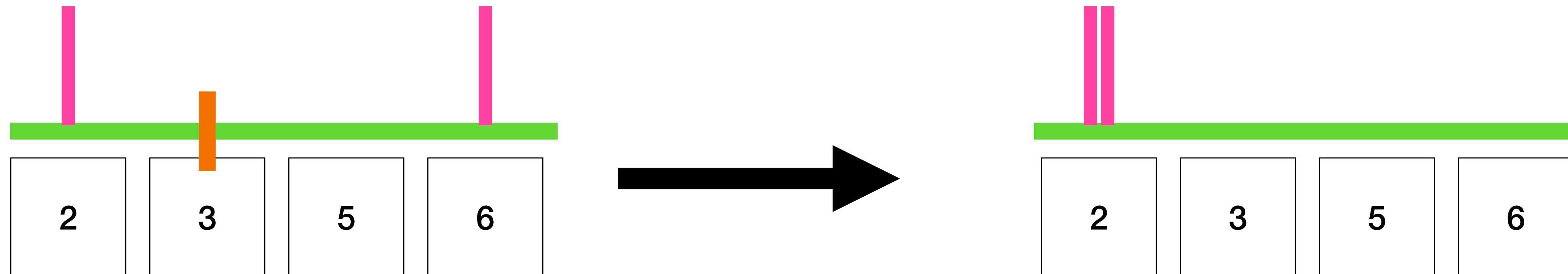
wir können aber feststellen, dass die $1 < 3$ ist. $\Rightarrow$ der **linke Teil ist für uns also interessant.**

Vergleiche Sparen durch Binary Search

Finden des Indexes zum Einsetzen

wir können aber feststellen, dass die $1 < 5$ ist. $\Rightarrow$ der **linke** Teil ist für uns **also interessant**.

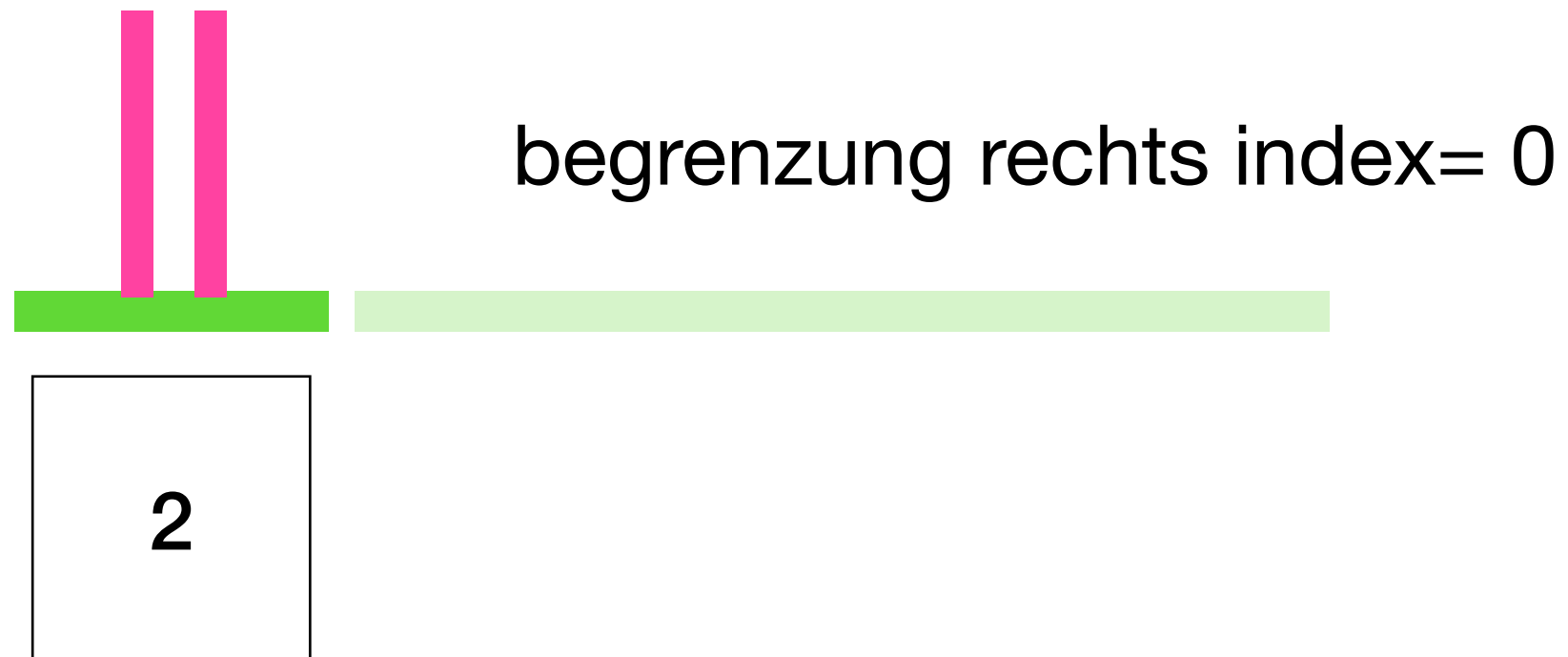
deshalb: Anpassen der **rechten** Grenze



wir müssen unsere Begrenzung nach rechts **links von der „Mitte“** platzieren.

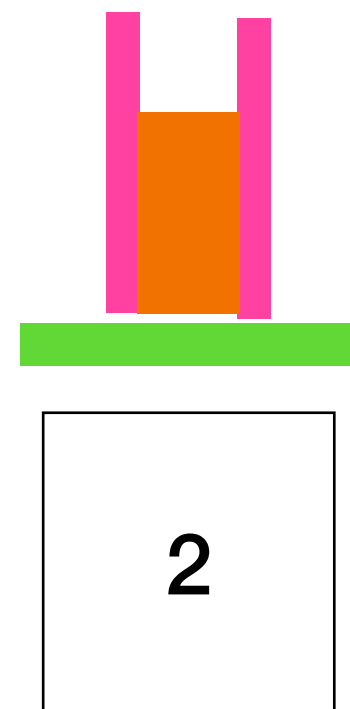
Vergleiche Sparen durch Binary Search

Finden des Indexes zum Einsetzen



die **neue Mitte** nehmen

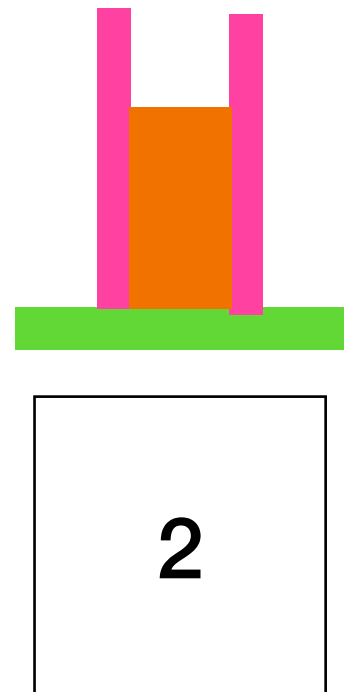
$$0+0//2 = \mathbf{0}$$



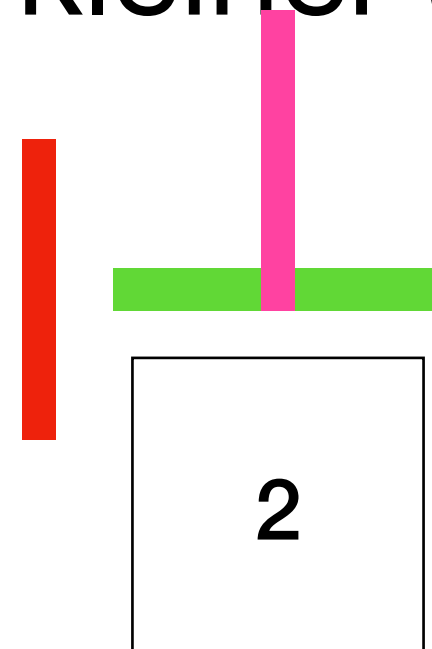
Vergleiche Sparen durch Binary Search

Finden des Indexes zum Einsetzen

die **neue Mitte** nehmen $0+0//2 = 0$

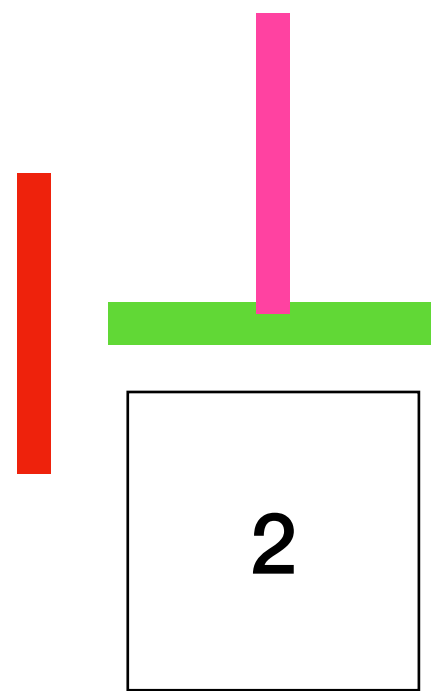


unser Wert ist immernoch kleiner als die Mitte ($1 < 2$)



Vergleiche Sparen durch Binary Search

Finden des Indexes zum Einsetzen



$i_rechts = -1$, $i_links = 0$

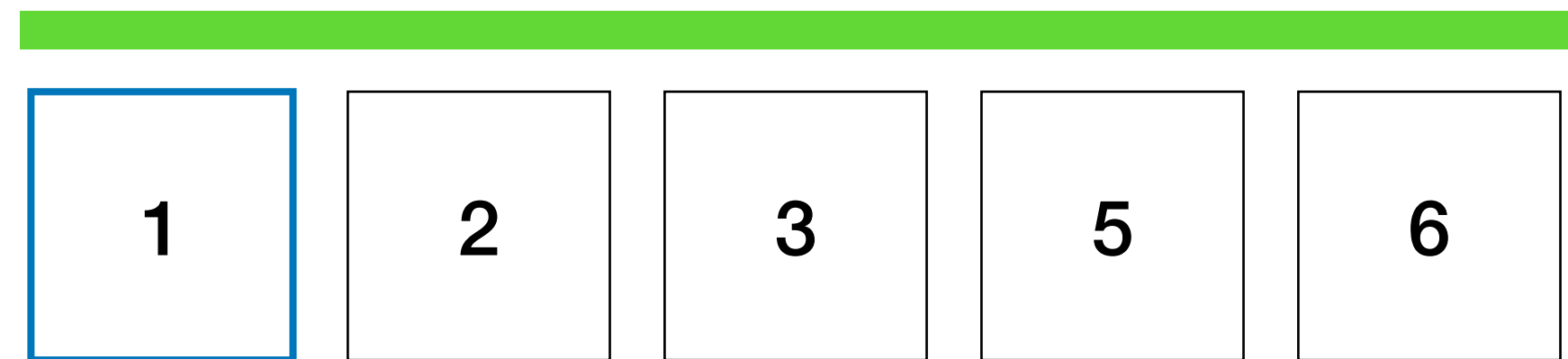
unser Wert ist immernoch kleiner als die Mitte ($1 < 2$)

rechte Grenze anpassen, die rechte Begrenzung steht nun links von der linken!!
($i_rechts < i_links$).

=> dies bedeutet wir sind Fertig

die 1 gehört an die **Position** der linken Begrenzung. $index = 0$

Das element einsetzen



bis zu diesem Punkt „bubbeln“ / swappen (nur ohne Vergleiche diesmal)

oder einsetzen, kann über *Liste.insert(index,wert)* gemacht werden

oder zusammenbasteln (*liste= slice1 + wert + slice2*)

Binärer Insertion vs. normaler Insertion

- **Normaler Insertion:**
 - Im schlimmsten Fall (umgekehrte Liste) für jedes neu einzusetzende Element durch den gesamten sortierten Teil vergleichen.
 - Dadurch im schlimmsten Fall asympt. n^2 Vergleiche.
- **Binärer**
 - wie **viele Vergleiche im schlimmsten Fall?**
-

Binärer Insertion vs. normaler Insertion

- **Normaler Insertion:**
 - Im schlimmsten Fall (umgekehrte Liste) für jedes neu einzusetzende Element durch den gesamten sortierten Teil vergleichen.
 - Dadurch im schlimmsten Fall asympt. n^2 Vergleiche.
- **Binärer**
 - wie viele Vergleiche im schlimmsten Fall?
 - **$n \cdot \log(n)$**

$n \cdot \log(n)$??

$$\sum_{i=1}^{n-1} a \cdot \log i = a \log((n-1)!) \in \Theta(n \log n)$$


log(i): „wie oft muss ich den sortierten Teil halbieren um zu wissen, wo mein Element hingehört“ (die Länge des sortierten Teils ist =i)

a: eine konstante, um zu skalieren, meistens nicht wichtig

Summe: Bei einer Liste mit Länge n muss ich $n-1$ Elemente einsortieren.
($n-1$, denn das erste war ja bereits sortiert, benötigt also keinen Vergleich)

Laufzeit von Rekursiven Funktionen

Übung 1

Geben Sie die Anzahl Aufrufe der Funktion f abhängig von n in der Θ -Notation für den untenstehenden Beispiel an. Kürzen Sie ihr Ergebnis soweit wie möglich. Geben Sie eine kurze Begründung.

```
1  # pre: n is an integer >= 0
2  def g(n):
3      count = 0
4      while n // (2 ** count) >= 1:
5          f()
6          count += 1
```


Antwort 1

```
1 # pre: n is an integer >= 0
2 def g(n):
3     count = 0
4     while n // (2 ** count) >= 1:
5         f()
6         count += 1
```

- Das Programm ruft die Funktion f , solange $n/2^{\text{count}} \geq 1$ ist.
- Diese Ungleichung gilt genau dann, wenn $\log n \geq \text{count}$.
- Die Variable *count* misst die Anzahl der Iterationen der while Schleife, und dabei auch die Anzahl Aufrufe der Funktion f .
- Darum ist diese Anzahl $\log n$.
- Das bedeutet, dass diese Anzahl $\Theta(\log n)$ ist.

Übung 2

Geben Sie die Anzahl Aufrufe der Funktion f abhängig von n in der Θ -Notation für den untenstehenden Beispiel an. Kürzen Sie ihr Ergebnis soweit wie möglich. Geben Sie eine kurze Begründung.

```
1  # pre: n is an integer >= 0
2  def g(n):
3      if n >= 1:
4          f()
5          g(n // 2)
```

Antwort 2

```
1  # pre: n is an integer >= 0
2  def g(n):
3      if n >= 1:
4          f()
5          g(n // 2)
```

Sei $T(n)$ die Anzahl der Aufrufe der Funktion f . Wenn $n \geq 1$, es gilt:
 $T(n) = 1 + T(n/2) = 1 + 1 + T(n/2^2) = k + T(n/2^k)$.

- f ist aufgerufen, solange $n/2^k \geq 1$ ist.
- Diese Ungleichung gilt genau dann, wenn $\log n \geq k$.
- Das bedeutet, dass die Anzahl der Aufrufe der Funktion f ist $\log n$ liegt, und dabei diese Anzahl $\Theta(\log n)$ ist.

Übung 3

Geben Sie die Anzahl Aufrufe der Funktion f abhängig von n in der Θ -Notation für den untenstehenden Beispiel an. Kürzen Sie ihr Ergebnis soweit wie möglich. Geben Sie eine kurze Begründung.

```
1  # pre: n is an integer >= 0
2  def g(n):
3      if n >= 1:
4          for i in range(n):
5              f()
6          g(n // 2)
```

... am besten wie vorhin die ersten paar summanden aufschreiben

Antwort 3

```
1 # pre: n is an integer >= 0
2 def g(n):
3     if n >= 1:
4         for i in range(n):
5             f()
6         g(n // 2)
```

Sei $T(n)$ die Anzahl der Aufrufe der Funktion f . Wenn $n \geq 1$, es gilt:

$$\begin{aligned} T(n) &= n + T(n/2) \\ &= n + n/2 + T(n/2^2) \\ &= n + n/2 + \dots + n/2^k + T(n/2^{k+1}) \\ &= n \sum_{j \leq k} 1/2^j + T(n/2^{k+1}) \\ &\leq 2n + T(n/2^{k+1}). \end{aligned}$$

- Die letzte Ungleichung gilt, weil $\forall k : \sum_{j \leq k} 1/2^j \leq 2$.
- Es gibt k , wofür $\lfloor n/2^{k+1} \rfloor = 0$, dann auch $T(\lfloor n/2^{k+1} \rfloor) = T(0) = 0$.
- Das bedeutet, dass $T(n) \leq 2n \in O(n)$.
- Analog dazu kann man zeigen, dass $T(n) \in \Omega(n)$, weil $g(n)$ ruft f zumindest n -mal.
- Darum, $T(n) \in \Theta(n)$.

Übung 4

Geben Sie die Anzahl Aufrufe der Funktion f abhängig von n in der Θ -Notation für den untenstehenden Beispiel an. Kürzen Sie ihr Ergebnis soweit wie möglich. Geben Sie eine kurze Begründung.

```
1  # pre: n is an integer >= 0
2  def g(n):
3      if n >= 1:
4          for i in range(n):
5              f()
6          g(n // 2)
7          g(n // 2)
```

Antwort 4

```
1 # pre: n is an integer >= 0
2 def g(n):
3     if n >= 1:
4         for i in range(n):
5             f()
6             g(n // 2)
7             g(n // 2)
```

Sei $T(n)$ die Anzahl der Aufrufe der Funktion f . Wenn $n \geq 1$, es gilt:

$$\begin{aligned} T(n) &= n + 2 T(n/2) \\ &= n + 2(n/2) + 2^2 T(n/2^2) \\ &= n + 2(n/2) + \dots + 2^k (n/2^k) \\ &\quad + 2^{k+1} T(n/2^{k+1}) \\ &= n (k + 1) + 2^{k+1} T(n/2^{k+1}). \end{aligned}$$

- f ist aufgerufen, solange $n/2^k \geq 1$ ist.
- Diese Ungleichung gilt genau dann, wenn $\log n \geq k$.
- Das bedeutet, dass $k = \log n$. Darum ist
$$T(n) = n (\log n + 1) + 2n T(1/2) = n (\log n + 1) + 2n T(0) = n (\log n + 1).$$
- Darum ist die Anzahl Aufrufe der Funktion f zwischen $\Theta(n \log n)$.

Master Theorem

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

wobei

n = **Problemgrösse**

a = **Anzahl der Teilprobleme** (wie oft ruft sich eine Funktion selbst auf)

b = **„Zerfalls Faktor“** der Problemgrösse

$f(n)$ = **Zusatzaufwand**, nicht rekursiv.

Master Theorem

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

wobei

n = **Problemgrösse** = n

a = (wie oft ruft sich eine Funktion selbst auf) = **2**

b = „**Zerfalls Faktor**“ der Problemgrösse = **2**

$f(n)$ = **Zusatzaufwand**, nicht rekursiv. = **n**

```
def g(n):  
    if n >= 1:  
        for i in range(n):  
            f()  
        g(n // 2)  
        g(n // 2)
```

2 calls

$$a \geq 1 \text{ und } b > 1$$

□ Master-Theorem – Kochrezept

$$T(n) = a \cdot T(n/b) + f(n)$$

$$a \geq 1, b > 1$$

1. $k = \log_b a$

2. Vergleiche $f(n)$ mit n^k :

Fall 1: $f(n) = O(n^{(k - \varepsilon)})$

$$\rightarrow T(n) = \Theta(n^k)$$

Fall 2: $f(n) = \Theta(n^k \cdot \log^p n)$

$$\rightarrow T(n) = \Theta(n^k \cdot \log^{(p+1)} n)$$

Fall 3: $f(n) = \Omega(n^{(k + \varepsilon)})$, Regularity erfüllt

$$\rightarrow T(n) = \Theta(f(n))$$

$$\log_2(2) = 1$$

□ Master-Theorem – Kochrezept

$$T(n) = a \cdot T(n/b) + f(n)$$

$$a \geq 1, b > 1$$

1. $k = \log_b a$

2. Vergleiche $f(n)$ mit n^k : $n^k = n^1 = n: \Rightarrow f(n) = n^k$

$$\log_2(2) = 1$$

Fall 1: $f(n) = O(n^{(k - \varepsilon)})$

$$\rightarrow T(n) = \Theta(n^k)$$

Fall 2: $f(n) = \Theta(n^k \cdot \log^p n)$

$$\rightarrow T(n) = \Theta(n^k \cdot \log^{(p+1)} n)$$

Fall 3: $f(n) = \Omega(n^{(k + \varepsilon)})$, Regularity erfüllt

$$\rightarrow T(n) = \Theta(f(n))$$

□ Master-Theorem – Kochrezept

$$T(n) = a \cdot T(n/b) + f(n)$$
$$a \geq 1, b > 1$$

1. $k = \log_b a$
2. Vergleiche $f(n)$ mit n^k :

$$\log_2(2) = 1$$

$$\Rightarrow f(n) = n^k$$

Fall 1: $f(n) = O(n^{(k - \varepsilon)})$
 $\rightarrow T(n) = \Theta(n^k)$

Fall 2: $f(n) = \Theta(n^k \cdot \log^p n)$
 $\rightarrow T(n) = \Theta(n^k \cdot \log^{(p+1)} n)$

p berechnen, damit die Gleichung erfüllt ist.

p=0

Fall 3: $f(n) = \Omega(n^{(k + \varepsilon)})$, Regularity erfüllt
 $\rightarrow T(n) = \Theta(f(n))$

Fall 2: $f(n) = \Theta(n^k \cdot \log^p n)$
 $\rightarrow T(n) = \Theta(n^k \cdot \log^{(p+1)} n)$

p=0 einsetzen.

$\Rightarrow$

$$T(n) = \Theta(n^1 \cdot \log^{0+1} n)$$

$$T(n) = \Theta(n \cdot \log^1 n)$$



Bemerkung

- Das Master Theorem wird vom Kurs aus nicht gelehrt, ist aber durchaus nützlich an der Klausur.
- Ich kann gerne bis zum Ende des Semesters noch genauer darauf eingehen für die anderen Fälle.

weiter mit: Merge Sort

Merge Sort

Idee

originale Liste

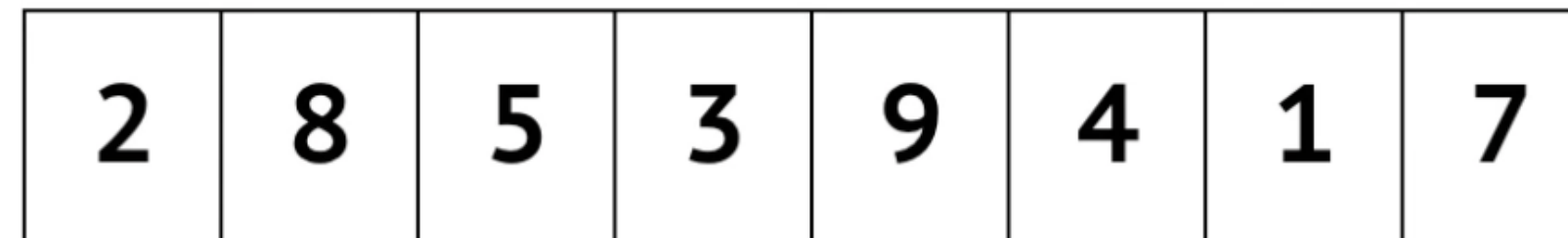
2	8	5	3	9	4	1	7
---	---	---	---	---	---	---	---

teilen

2	8	5	3	9	4	1	7
---	---	---	---	---	---	---	---

Merge Sort

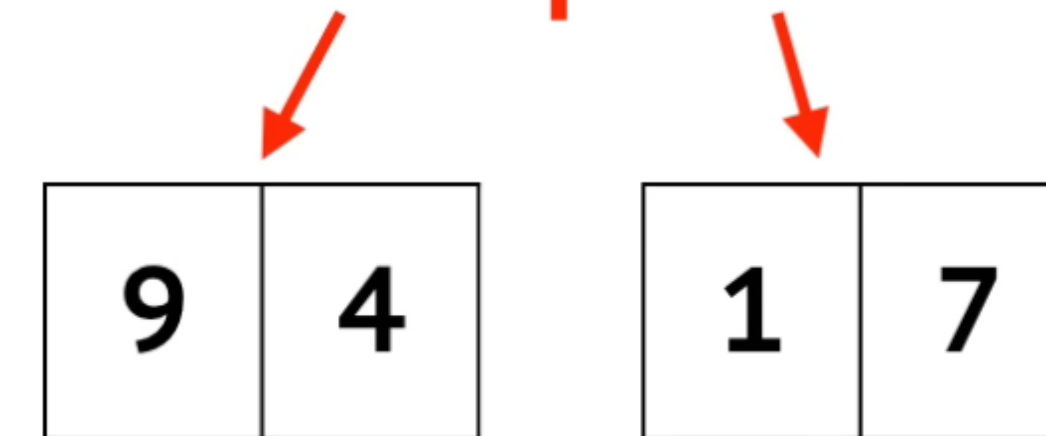
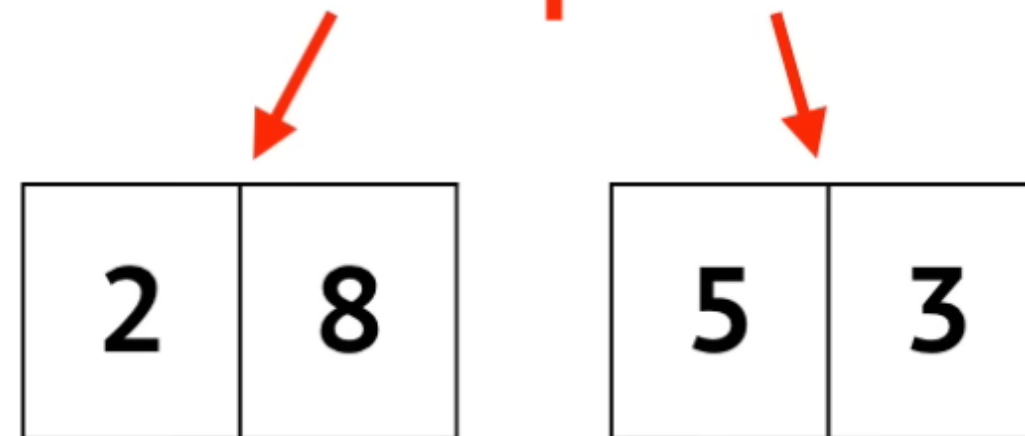
Idee



teilen



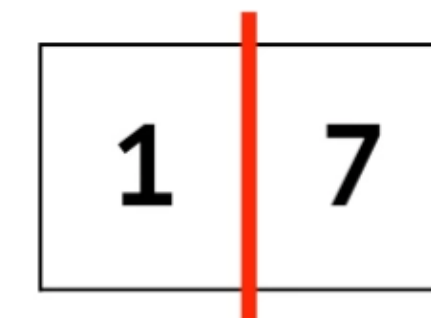
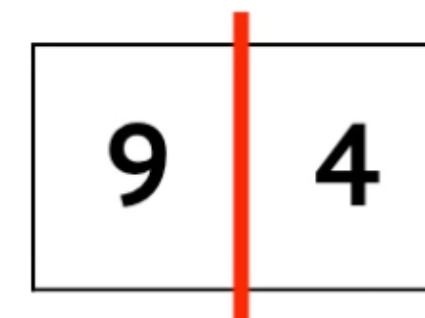
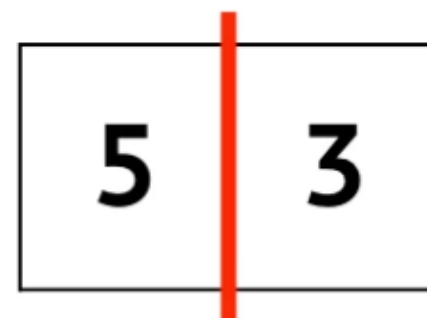
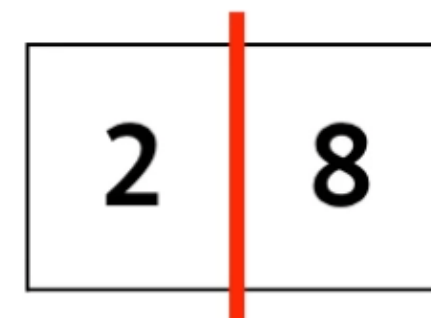
nochmals teilen



und nochmals...

Merge Sort

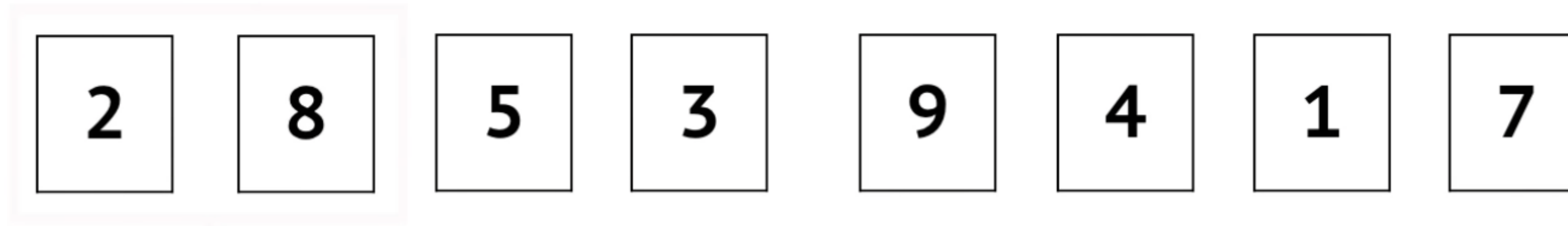
Idee



nun haben wir alle Elemente einzeln.

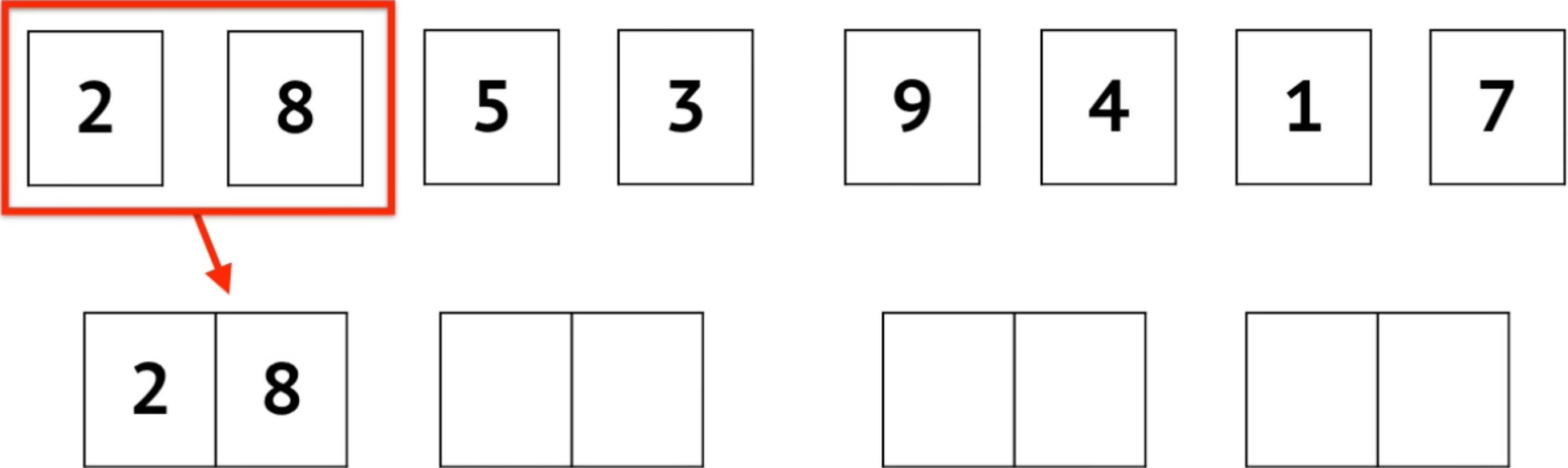
* im code würde die Liste nicht exakt in dieser Reihenfolge geteilt werden, aufgrund der Rekursion, aber das folgende Prinzip ist dasselbe.

Das Mergen im Merge-Sort



Elemente liegen alle einzeln vor

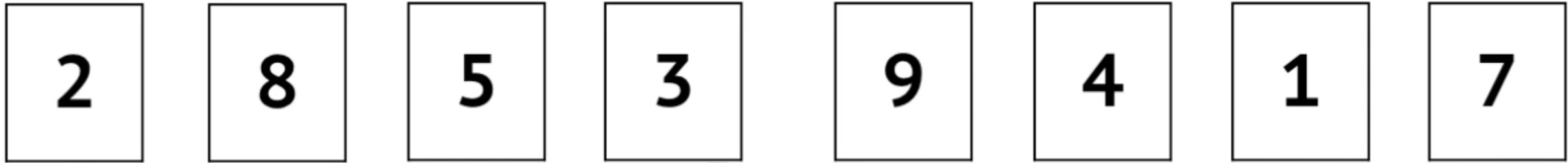
Das Mergen im Merge-Sort



Elemente liegen alle einzeln vor

Paarweise sortieren.

Das Mergen im Merge-Sort

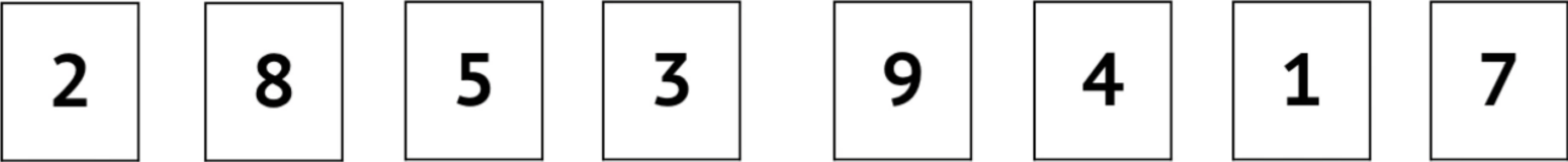


Elemente liegen alle einzeln vor

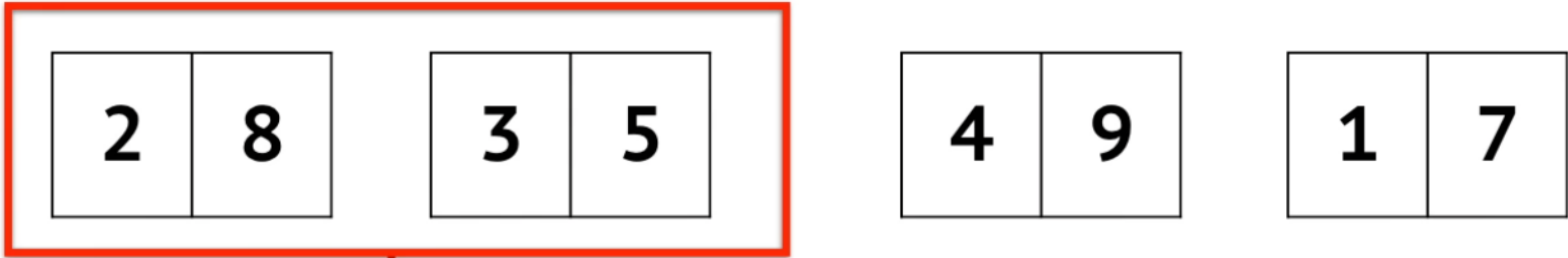


Paarweise sortieren.

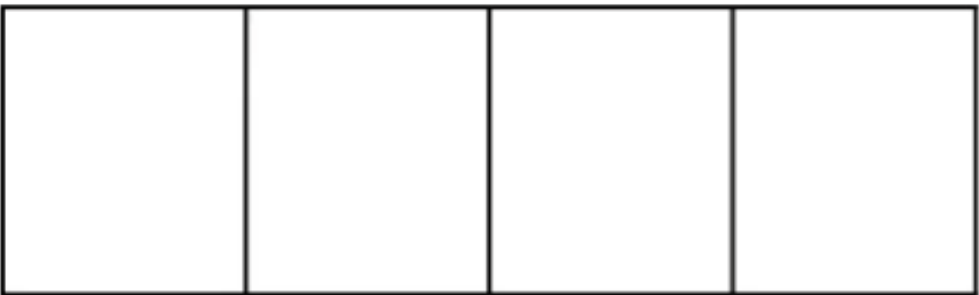
Das Mergen im Merge-Sort



Elemente liegen alle einzeln vor

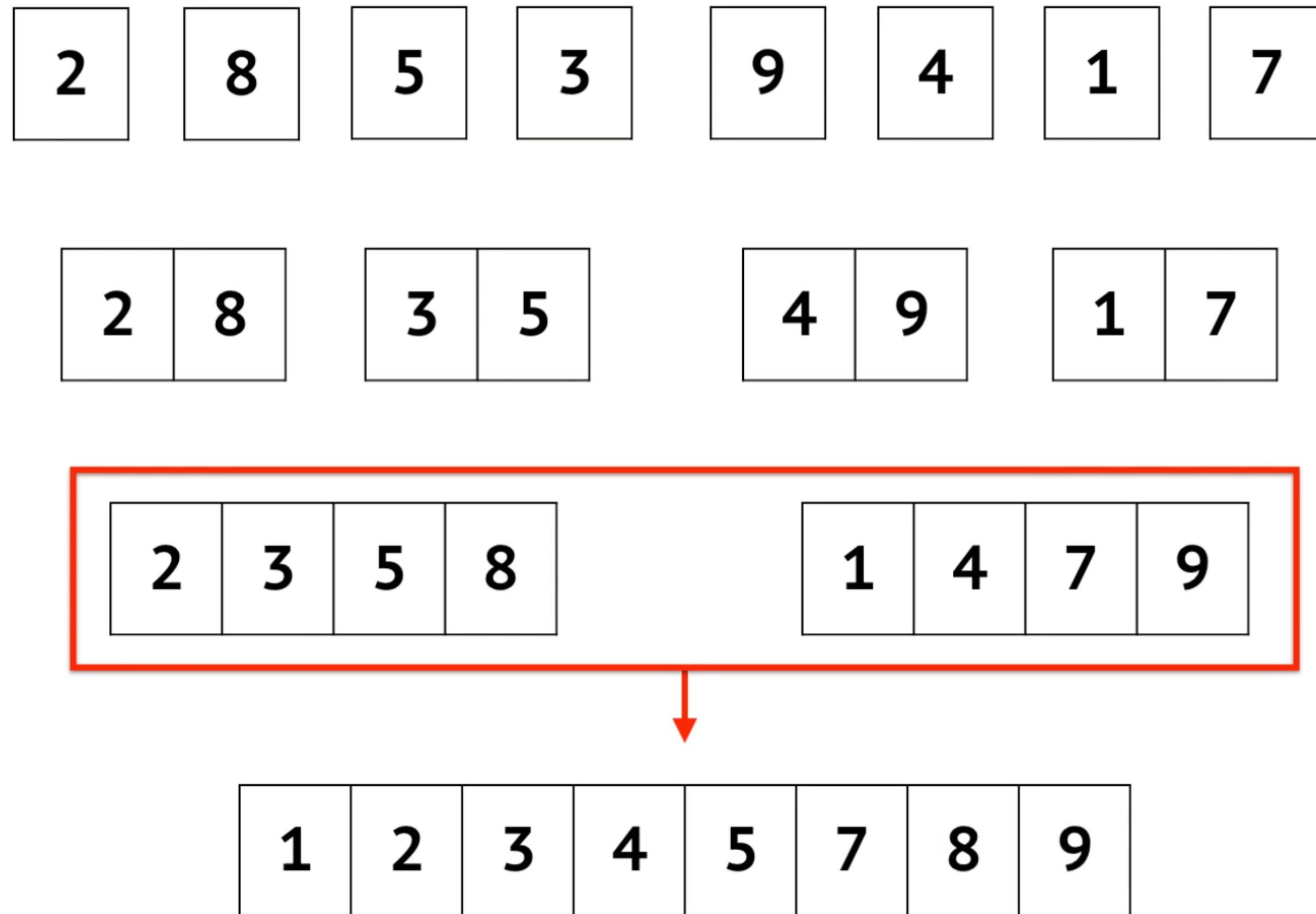


Paarweise sortieren.



selbes Spiel.

Das Mergen im Merge-Sort



Grafik von Michael Sambol auf YT. **Empfehlung für die Lernphase!**, Er hat jeden wichtigen SortAlg erklärt.

Der code

zwei Funktionen:

- eine macht nur das Merging.
- Die andere macht **Rekursion**, das heisst:
 - sie teilt die Liste rekursiv
 - danach merged sie bottom up. mergen

wir schauen uns das .ipynb an.

...dort können wir auch direkt die Reihenfolge des teilens anschauen

Exercise 6: Search & Sort Fällig 6.April

Bemerkungen& Tipps

- **Eierwerfen:** reine Gedankenübung
- **Vergleich Sortieralgorithmen:** Systematisch vorgehen.
- **Verbesserter InsertionSort:** Slides anschauen, im Code umsetzen.
- **Invarianten Suchalgorithmen:** Schritt für Schritt durchgehen der Algorithmen

schöne Ferien

Anonymes Feedback Formular

